

Jes
Ak
M.
En
(IC
20
Ns

**CLASSIFICATION OF EARLY AND LATE BLIGHT DISEASE OF POTATO
USING CONVOLUTIONAL NEURAL NETWORK**

Jesmin Akther

MASTER'S OF SCIENCE ON ENGINEERING

NOAKHALI SCIENCE AND TECHNOLOGY UNIVERSITY

**CLASSIFICATION OF EARLY AND LATE BLIGHT DISEASE OF POTATO
USING CONVOLUTIONAL NEURAL NETWORK**

Jesmin Akther

**Thesis submitted in fulfillment of the requirements for the award of the degree
of masters of science of Engineering in in Information and Communication
Engineering.**

Department of Information and Communication Engineering.

NOAKHALI SCIENCE AND TECHNOLOGY UNIVERSITY

STUDENTS DECLARATION

I hereby declare that the work in this thesis is my own except for quotations and summaries which have been duly acknowledged. The thesis has not been accepted for any degree and is not concurrently submitted for award of other degree.

Jesmin Akther

Signature

Name: Jesmin Akther.

ID Number: BKH1711MSc114F

Date: 23/06/2020

SUPERVISOR DECLARATION

I, hereby declare that I have checked this thesis and in my opinion, this thesis is adequate in terms of scope and quality for the award of the degree of Master of engineering of science in Information and Communication Engineering.

Signature

Name of Supervisor: Dr. Md. Ashikur Rahman Khan

Position: Professor

Dept. of ICE.

Date:

Signature

Name of Supervisor: Sultana Jahan Soheli

Position: Assistant Professor

Date:

ACKNOWLEDGEMENT

I am grateful and indebted to my supervisor Professor Dr. Md. Ashikur Rahman Khan for his invaluable guidance, continuous encouragement and constant support in making this research possible. I appreciate his consistent support from the first day I applied to graduate program to these concluding moments. In true sense, he is a friend, philosopher and guide. Even volumes of words will fall short in thanking him for the kind of support and guidance given by him.

I also would like to express very special thanks to my co-supervisor Assistant Professor Sultana Jahan Soheli for her suggestions and co-operation throughout the study. I also sincerely thanks for the time spent proofreading and correcting my many mistakes.

My sincere thanks go to all my members and staff of the Information and Communication Engineering Department, NSTU, who helped me in many ways and made my stay at NSTU pleasant and unforgettable. Many special thanks go to member programming research group for their excellent cooperation, inspirations and supports during this study.

I acknowledge my sincere indebtedness and gratitude to my parents for their love, dream and sacrifice throughout my life. Special thanks should be given to my mother who helps me a lot data research and study. I would like to acknowledge her suggestions which was crucial for the successful completion of this study.

ABSTRACT

Crop diseases are a major threat to cultivate, need to supervise growth and detrimental diseases early or proper detection. The potato crop is a vital among the foods around so many countries with high potential value and produced in large quantity. Potato early and late blight disease symptoms and detection both are vague to distinguish and isolation. The rising combination of smartphone penetration and recent advances deep learning has

paved the way for smart device assisted disease prognosis. Relying on pure naked-eye observation to detect and classify diseases can be very cumbersome. So absolute detection process proves to be effective and convenient for researchers. The color and analyze layer features are used to best match to recognize and classify different agriculture produce into early blight and late blight affected disease. Both features prove to be very effective in disease detection. The experimental results indicate that proposed approach significantly enhances accuracy in automatic detection of both affected blight. Deep learning is a part of machine learning where methods are based on artificial neural networks. This paper proposed a convolutional neural network model called Sequential to detect and identify diseases in potato leaves labeling early and late blight. In addition to normalization, divides and extracted the images to prepare data prior convolutional neural network (CNN). This trained model assures 94% accuracy and short timing classification of both early blight and late blight. The technique of training deep learning models on increasingly vast and globally available and image datasets presents a clear path toward smartphone aid crop disease experiment on around the world.

TABLE OF CONTENTS

TITLE	PAGE
TITLE PAGE	ii
COVER PAGE	iii
STUDENT DECLARATION	vi
SUPERVISOR DECLARATION	v
ACKNOWLEDGEMENT	vi
ABSTRACT	vii
TABLE OF CONTENTS	viii
LIST OF TABLES	xiii
LIST OF FIGURES	xiv
LIST OF SYMBOLS	xvi

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

xvii

CHAPTER 1 INTRODUCTION

1.1 Introduction 1

1.2 Statement of Problem 3

1.3 Objectives 3

1.4 Scope of the Research 4

1.5 Overview of the Thesis 5

CHAPTER 2 LITERATURE REVIEW

2.1 Introduction 6

2.2 CNN Analysis 6

2.3 Convolutional layer 7

2.3.1 Depth 7

2.3.2 Stride 7

2.4 Pooling layer 8

2.5 ReLU activation 8

TABLE OF CONTENTS

2.6 Fully connected layer	9
2.7 Receptive field	9
2.8 Weights	9
2.9 Hyper parameters Selection	10
2.9.1 Number of filters	10
2.9.2 Feature maps	10
2.9.3 Filter shape	10
2.10 Regularization methods	11
2.10.1 Dropped out	11
2.10.2 Drop-Connect	11
2.10.3 Stochastic pooling	11
2.10.4 Number of parameters	12
2.10.5 Weight decay	12
2.11 Performance Modelling	12
2.11.1 Hyper-parameter tuning	13

TABLE OF CONTENTS

2.11.2 Data redesigning	13
2.11.3 Model optimization	14
2.12 Improving Performance of CNN	14
2.12.1 Tune Parameters	14
2.12.2 Image Data Augmentation	15
2.12.3 Deeper Network Topology	15
2.12.4 Handel Overfitting and Underfitting problem	15
2.13 Literature Review	16
2.14 Conclusion	19
 CHAPTER 3 METHODOLOGY	
3.1 Introduction	20
3.2 Research Design	20
3.3 Sampling	22
3.4 Instrument	24
3.5 Steps for Proposed Method	26

TABLE OF CONTENTS

3.5.1 Raw Leaves collection	26
3.5. 2 Image Process	28
3.5. 3 Data Storage	29
3.5.4 Data preprocess	30
3.5. 5 Train Data	30
3.5. 6 Deploy CNN and Analyze model	32
3.5.7 Optimize and Build Model	37
3.5.8 Classification	39
3.6 Conclusion	40
 CHAPTER 4 RESULT AND DISCUSSION	
4.1 Introduction	41
4.2 Pre-process Output	42
4.3 Train Data Output	43
4.4 First Deploy CNN	43
4.5 Analyze and Optimize model	48
4.5.1 Analyze with different size model	48

TABLE OF CONTENTS

4.5.2 Full model analyze	51
4.6 Experiment Result	54
4.7 Model Test Overview	56
4.8 Data Preparation	57
4.9 Step-by-Step Method	57
4.10 Conclusion	58

CHAPTER 5 CONCLUSION

5.1 Introduction	59
5.2 Discussion	59
5.3 Future work	60
REFERENCES	61
APPENDICES	
A Collected Data	67

TABLE OF CONTENTS

B Coding	68
-----------------	----

LIST OF TABLES

Table No	Title	Page
3.1	Three best layers table.	38
3.2	Model Parameter table.	39
4.1	Final accuracy compilation.	55
4.2	Five best layer result.	56

LIST OF FIGURES

Figure No	Title	Page
1.1	Effectuated potatoes and potato crops	2
3.1	Basic steps for disease detection.	21
3.2	Effectuated late blight symptoms on stem and leaf	22
3.3	Effectuated early blight symptom on stem and leaf	23
3.4	Affected potatoes early and late blight	24
3.5	Proposed Methodology	26
3.6	Field survey of potato leaves.	27
3.7	Affected Late Blight and Early Blight potato with deeper severity	18

Figure No	Title	Page
3.8	Resize Image Array	30
3.9	Train Data Time Speed	31
3.10	Image pre-process Implement-View.	33
3.11	Conv2D parameter filters.	34
3.12	MaxPooling	34
3.13	A small design of CNN	37
4.1	Image channel modification.	42
4.2	Train data with speed.	43
4.3	Deployed CNN overview	44
4.4	First CNN Compilation.	45
4.5	Data Accuracy and Loss graph	46
4.6	Epoch Accuracy and Loss with the Tensor Board.	47
4.7	A small model implementation summery	49
4.8	Small model with val_accuracy	50
4.9	Full update model implementation summery.	51
4.10	Full update model compilation	52

Figure No	Title	Page
4.11	Full model train and validation accuracy Plot.	53
4.12	Testing Process	57

LIST OF SYMBOLS

xx	feature
yy	label
ACC	Accuracy
VAL_ACCURACY	Validation Accuracy
VAL_LOSS	Validation_LOSS

LIST OF ABBREVIATIONS

JPEG	Joint Picture Expert Group
CNN	Convolutional Neural Networks
SVM	Support Vector Machine
MLP	<u>Multilayer perceptron</u>
ConvPool	Convolution and Pooling
ReLU	Rectified linear unit
GPU	Graphics Processing Unit
Colab	Google Colaboratory
NN	Neural Network
DCNN	Deep Convolutional Neural Networks
API	Application Programing Interface

CHAPTER 1

INTRODUCTION

1.1 Background

This chapter discuss about the importance of potato cultivation, their disease symptoms and related work that already applied for disease detection. Bangladesh is export potato which is world's fourth largest food crop after maize, wheat, and rice. Potato export reached a record high of over 100,000 tonnes in 2013-14 also plummeting to 53,000 tonnes last few years. It was 27,811.6 tonnes till April 2019. The Russian Federal Service for Veterinary and Phytosanitary Surveillance said the potatoes from Bangladesh were infected by brown rot disease and potato tuber moth, as there is no accredited or standard lab in Bangladesh to test the quality of potatoes found by Molla and Zahangir [1] [2]. Including brown rot disease and potato tuber moth diseases, there are so many disease of

potatoes such as potato late blight and potato early blight are detrimental both of crops growth and glut production. Disease symptoms occur at any age of plants including yellowing leaves, stunting and wilting plants Author [3]. Potato early blight and late blight are difficult to detect and prevent on naked eye. Identify such disease in manually is tough for farmers in consequences hinder glut growth of crops lack of proper steps. Late blight is the most common and highly destructive, fungal disease in potato, tomato and other solanaceae crops in Bangladesh. Disease appears to be severe to succulent plants by wilting of leaves and collapse of stems. Annual potato yield losses due to late blight have been estimated at 25-57%. Late blight can be controlled applying costly fungicides. Moreover, the degree of control heavily depends on the timing of the fungicide application in relation to local weather conditions, crop development and disease pressure. So, the efficiency of late blight control can be improved considerably by informing farmers in time about predicted infection periods of the potato crop and the effectiveness of past spray applications illustrate by Kessel [5]. Trials at Michigan State University (MSU) in 2013 indicated that Ridomil applied curatively to blighted foliage did not successfully prevent further disease development Control of foliar late blight with fungicides in 2013 at the Clarksville Research Station [6].Extracted data full report on page 138 of the first symptoms of early blight appear as small, circular or irregular, dark-brown to black spots on the older leaves. The affected potato and potato crops in early blight and late blight diseases is shown in Fig. 1.1.



a. Early blight effected potatoes (b) Effected potato crops



(c) Late blight effected potatoes (d) Effected potato crops

Figure 1.1. Effected potatoes and potato crops

These spots enlarge up to 3/8 inch in diameter and gradually may become angular-shaped (Figure 1.1 b). Late blight damages leaves, stems and tubers shows on figure 1.1(c). Affected leaves eventually rot and dry out. When drying out, leaves turn brown or black in color. When active infections, spots appear on the underside of leaves blanketed in what looks like flour. Affected stems begin to blacken from their tips, and eventually dry out. Severe infections cause all foliage to rot, dry out and fall to the ground, stems to dry out and plants to die. Affected tubers display dry brown-colored spots on their skins and flesh. This disease acts very quickly. If it is not controlled, infected plants will die within two or three days [7].

1.2 Statement of Problem

Plant diseases Identification is the key to preventing the losses in the yield and quantity of the agricultural product. The studies of the plant diseases mean the studies of visually observable patterns seen on the plant. Health monitoring and disease detection on plant is very critical for sustainable agriculture. It is very difficult to monitor the plant diseases manually. The diagnostician must have very good observation skills, and needs to be a good detective. Without proper identification of the disease and the disease-causing agent, disease control measures can be a waste of time and money and can lead to further plant losses. Identification of affected plants is one of the first steps in diagnosing a plant disease. Proper disease diagnosis is therefore essential. Often, plant pathologists have to rely on symptoms for the identification of a disease problem. It requires tremendous amount of work, expertise in the plant diseases. Manually disease detection complex in nature and is a time consuming process. Also requires the excessive processing time. Recurrently ranchers are maladroitness to virtual pattern detection that leads inaccurate imagination. Due to misconstrue intensity of the disease, pesticide's under-dosage or over-dosage retract healthy crops. Therefore need a soothe manual disease prediction through boon of methodology of deep learning along minimal error.

1.3 Objectives

The objective for potato disease are pointed below:

- a. Plant pathologists desire an accurate and reliable plant disease diagnosis system where identify disease by smartphone pave the way.
- b. Soothe manual assumption by get to know the severity of the disease.
- c. To finalize a model that can be fitted various application for further utilize.
- d. To predict and analyze the profit observing growth of plants and minimal cost with a user friendly research field..
- e. Earlier potato blight disease detection with prevention using Retrieval and Classification
- f. Plant disease Quantification and Expert System

1.4 Scope of the Research

Advance technology could be ensure disease strains as early and as quickly as possible to protect crop yields and reduce dependency on pesticides. A vital step in inscribe potato crop disease is the implementation of adequate surveillance strategies so that rapid, in field diagnosis can be made. The diagnostic system will deliver in-field results in minutes while current lab tests can take up to one week. Diseased plants lead to declassification or

even rejection of the seed lots resulting in a financial loss shown by authors Gerrit Polder et al [8]. Farmers put in a lot of effort to detect diseased plants and remove diseased plants from the field. Nevertheless, dependent on the cultivar, diseased plants can be missed during visual observations in particular in an early stage of cultivation. Therefore, there is a need for fast and objective disease detection. Early detection of diseased plants with modern vision techniques can significantly reduce costs.

1.5 Overview of the Thesis

Chapter 1 gives the introduction covering brief account of plant disease, Problem and difficulties and contribution of information technology in agriculture. In chapter 2 practical and theoretical concepts of disease detection and classification method are discussed. The limitation and difficulties of classification and detection method are explained. Brief analysis of two potato disease problems are identified. Chapter 3 deals with the experiment layout of potato disease detection. The main motive of this chapter is to find out the infected lesion in potato leaves. Infected lesion is found with the help of image pre-processing background separation and color filtering operation. Then prepare data to train dataset and analyze and optimize dataset for classify dataset. Chapter 4 the classification of disease is discussed and developed model. The train model discriminate analysis based classification method is used for identification of disease and the result are quite accurate. Last chapter 5 conclusion and future of this research are enumerated in detail. In the next chapter related word with various way to crop disease detection, boon of deep learning role in advance era are reviewed. Mobile assisted different algorithm are discuss for reviewing related work. Limitation and proposed research are discuss in literature review chapter.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Agricultural crop have become a beneficial resource of power energy and also mystery to resolve the difficulty of global warming. There are various disease that influence agriculture plants cause to disturbing cost-effective, environmental losses. There are numerous ways to identify plant disease. Generally various kind of laboratory analysis, usually by way of powerful and accurate microscope, is compulsory. With increasing penetration of internet and smart devices this is possible using remote sensing technique remote disease detection through image process capture by multi-hyperspectral digital image captures. Digital image processing helps to solve many problems or reduce by use of pattern recognition and statistical analysis tools. In chapter two various technique are discussed for plant disease detention and classification. By monitoring service for disease control and cure in early stage. The timely recognition of potato plant infection by disease or other could be helpful and valuable source of information for performing the exact pest management with cure and infection, manage measures to avoid the growth and the spread of potato harmful diseases. The chapter is divided e.g. potato blight disease, disease sensing, Disease detection method, disease classification.

2.2 CNN Analysis

A CNN consists of an input and an output layer, as well as multiple hidden layers. The hidden layers typically consist of a series of convolutional layers that convolve with a multiplication or other dot product. The activation function is commonly a rectified linear unit (RELU) layer, and

is subsequently followed by additional convolutions such as pooling layers, fully connected layers and normalization layers, referred to as hidden layers because their inputs and outputs are masked by the activation function and final convolution. Mathematically, convolution is technically a sliding dot product or cross-correlation.

2.3 Convolutional layer

The convolutional layer is the core building block of a CNN. The layer's parameters consist of a set of learnable filters or kernels, which have a small receptive field, but extend through the full input volume. During the forward pass, each filter is convolved across the width and height of the input volume, computing the dot product between the entries of the filter and the input and producing a 2-dimensional activation map of that filter. As a result, the network learns filters that activate when it detects some specific type of feature at some spatial position in the input. A convolutional layer within a neural network should have the following attributes. Three hyper parameters control the size of the output volume of the convolutional layer: the depth, stride and zero-padding. Generally, the convolutional layer output can be represented by Eq. (1)

$$M_j^P = f(\sum_{i \in M_j} M_i^{p-1} * k_{ij} + N_j^p)$$

(1)

where p represents the pth layer, k_{ij} denotes convolutional kernel, N_j denotes bias and M_j denotes a set of input maps.

2.3.1 Depth

For example, if the first convolutional layer takes the raw image as input, then different neurons along the depth dimension may activate in the presence of various oriented edges, or blobs of color.

2.3.2 Stride

Stride controls how depth columns around the spatial dimensions (width and height) are allocated. When the stride is 1 then we move the filters one pixel at a time. This leads to heavily overlapping receptive fields between the columns, and also to large output volumes. In practice, stride lengths of stride>3 are rare. The spatial size of the output volume can be computed as a function of the input volume size W, the kernel field size of the convolutional layer neurons K, the stride with which they are applied S, and the amount of zero padding P used on the border.

The formula for calculating how many neurons "fit" in a given volume is given by Eq. (2)

$$1 + \frac{W - k + 2p}{S}$$

(2)

2.4 Pooling layer

The pooling layer operates independently on every depth slice of the input and resizes it spatially. The most common form is a pooling layer with filters of size 2×2 applied with a stride(S) of 2 down samples at every depth slice in the input by 2 along both width(x) and

height(Y), discarding 75% of the activations. Pooling layer output can be represented by Eq. (3)

$$f_{X,Y}(S) = \max_{a,b=0}^1 S^2 x a + a, 2Y + b$$

(3)

2.5 ReLU activation

ReLU layer applies the non-saturating activation function $f(x) = \max(0, x)$. It effectively removes negative values from an activation map by setting them to zero. Other functions are saturating hyperbolic tangent $\{ \displaystyle f(x) = \tanh(x) \}$ $f(x) = \tanh(x)$, $f(x) = |\tanh(x)|$ and the sigmoid function $\sigma(x) = (1 + e^{-x})^{-1}$. $\{ \textstyle \sigma(x) = (1 + e^{-x})^{-1} \}$ ReLU is often preferred to other functions because it trains the neural network several times faster without a significant penalty to generalization accuracy.

2.6 Fully connected layer

Fully connected layers connect every neuron in one layer to every neuron in another layer. It is in principle the same as the traditional multi-layer perceptron neural network (MLP). The flattened matrix goes through a fully connected layer to classify the images.

2.7 Receptive field

In a convolutional layer, neurons receive input from only a restricted subarea of the previous layer. Typically the subarea is of a square shape size 5 by 5. The input area of a neuron is receptive field. So, in a fully connected layer, the receptive field is the entire previous layer. In a convolutional layer, the receptive area is smaller than the entire previous layer.

2.8 Weights

The function that is applied to the input values is determined by a vector of weights and a bias typically real numbers. Learning, in a neural network, progresses by making iterative adjustments to these biases and weights.

2.9 Hyper parameters Selection

CNNs use more hyper parameters than a standard multilayer perceptron (MLP). While the normal rules for learning rates and regularization constants still apply, the following should be kept in mind when adjusting.

2.9.1 Number of filters

Since feature map size decreases with depth, layers near the input layer will tend to have fewer filters while higher layers can have more. To equalize computation at each layer, the product of feature values with pixel position is kept roughly constant across layers. Preserving more information about the input would require keeping the total number of activations non-decreasing from one layer to the next.

2.9.2 Feature maps

The number of feature maps directly controls the capacity and depends on the number of available examples and task complexity.

2.9.3 Filter shape

Common filter shapes found in the literature vary greatly, and are usually chosen based on the dataset. There is a challenge to find the right level of granularity so as to create abstractions at the proper scale, given a particular dataset, and without overfitting.

2.10 Regularization methods

Regularization is a process of introducing additional information to solve an ill-posed problem or to prevent overfitting. CNNs use various types of regularization.

2.10.1 Dropped out

At each training stage, individual nodes are either "dropped out" of the net with probability $1-p$ or kept with probability p , so that a reduced network is left; incoming and outgoing edges to a dropped-out node are also removed. Only the reduced network is trained on the data in that stage. The removed nodes are then reinserted into the network with their original weights. The biggest contribution of the dropout method: although it effectively generates $2^n \times 2^n$ neural nets, and as such allows for model combination, at test time only a single network needs to be tested.

2.10.2 Drop-Connect

Drop Connect is the generalization of dropout in which each connection, rather than each output unit, can be dropped with probability $1-p$. Each unit thus receives input from a random subset of units in the previous layer.

2.10.3 Stochastic pooling

A major drawback to Dropout is that it does not have the same benefits for convolutional layers, where the neurons are not fully connected. In stochastic pooling, the conventional deterministic pooling operations are replaced with a stochastic procedure. This approach is free of hyper parameters and can be combined with other regularization approaches, such as dropout and data augmentation.

2.10.4 Number of parameters

This is a simple way to prevent overfitting is to limit the number of parameters. For convolutional networks, the filter size also affects the number of parameters. Limiting the number of parameters restricts the predictive power of the network directly. Reducing the complexity of the function that it can perform on the data, and thus limits the amount of overfitting.

2.10.5 Weight decay

A simple form of added regularizer is weight decay, which simply adds an additional error, proportional to the sum of weights or squared magnitude of the weight vector, to the error at each node.

2.11 Performance Modelling

Performance Modelling Increasing or decreasing contains theoretical dimensional shaping with CNN. Increasing or decreasing the number of new convolutional layers, Increasing or decreasing the number of nodes in each new convolutional layer. Tuning hyper-parameters like activation functions and learning rates. Experimenting with the image preprocessing you apply. Unfreezing the fully-connected layer and adjusting/training that

as well. Unfreezing one or more preexisting convolutional layers and (re-)training those as well. The best performing code implemented using ImageDataGenerator, flow from data frame, various data augmentation techniques, L2 regularization, batch normalization, one hot encoding on a subset of data considering the class weights in Keras. Keras library easy to work with python where background Tensorflow. Different technique's for performance are mention here.

2.11.1 Hyper-parameter tuning

Hyper-parameter tuning refers to different types of function activation and layers. Namely function are ReLU, sigmoid, Tanh Softmax Dense layer. Apart them epochs, learning rate increase model performance with overfitting and underfitting.

- **Regularization**

Penalizing complexity of the model, penalizing big weights. Regularization encourage weights to be small but doesn't force them to exactly 0.

- **Learning Rate (LR) optimization**

Starting with a base LR and subsequently decreasing it for next epoch.

- **Batch Size**

Usually try with max batch size your GPU can handle, depends on the memory of the GPU.

- **Increase model capacity**

Increasing model depth that is more number of layers and width (number of filters in each convolution layer).

2.11.2 Data redesigning

- **Progressive resizing:**

From 128 x 128 x 3 to 256 x 256 x 3 or to even higher size.

- **Random image rotations:**

Change orientation of the image.

- **Random image shifts:**

Useful when object is not at the center of the image.

- **Vertical and horizontal shift:**

Randomly flip the image vertically or horizontally. Algorithm should identify a glass whether it's face up or face down

2.11.3 Model optimization

- **Fine tuning the model with subset data:**

Dropping few data samples for some of the overly sampled data classes.

- Class weights:

Used to train highly imbalanced (biased) database, class weights will give equal importance to all the classes during training.

- Fine tuning the model with train data:

Use the model to predict on training data, retrain the model for the wrongly predicted images.

2.12 Improving Performance of CNN

CNN among algorithms of deep learning has been one of the most influential innovations in the field of deep learning. To improving performance certain ideas to lift the performance of CNN such as Tune Parameters, Image Data Augmentation, Deeper Network Topology

And Handel Overfitting and Underfitting problem.

2.12.1 Tune Parameters

In section 2.11.1 Tune Parameters are discussed. Number of epochs definitely affect the performance. But need to do certain experimentation for deciding epochs, learning rate. Decision could make after certain epochs if there is not any reduction in training loss and improvement in training accuracy. Also dropout layer in the CNN model can be used and proper optimizer during compilation of model. Various optimizer e.g. Adam, SGD, rmsprop these things affect the performance of CNN.

2.12.2 Image Data Augmentation

CNN requires the ability to learn features automatically from the data, which is generally only possible when lots of training data is available. Image augmentation parameters that are generally used to increase the data sample count are zoom, shear, rotation, preprocessing function and so on. Definitely it will increase the accuracy of system.

2.12.3 Deeper Network Topology

Deeper networks capture the natural “hierarchy” that is present everywhere in nature. See a convnet for example, it captures low level features in first layer, a little better but still low level features in the next layer and at higher layers object parts and simple structures are captured. The advantage of multiple layers is that they can learn features at various levels of abstraction. The wider network will take longer time to train. Deep networks are very computationally expensive to train. Hence, make them wide and deep enough that they work well.

2.12.4 Handel Overfitting and Underfitting problem

- Overfitting refers to a model that models the training data too well. The Example of underfitting is your model is giving 50% accuracy on train data and 80% accuracy on test data. The meaning of it, In the overfitting your model gives very nice accuracy on trained data but very less accuracy on test data. Overfitting model is having good memorization ability but less generalization ability. This also refers model doesn't generalize well from our training data to unseen data.

- Underfitting refers to a model which works unwell on train as well as test data. Its very dangerous. The Example of overfitting is your model is giving 99% accuracy on train data and 60% accuracy on test data. Model is not fitted well on training data itself. There are certain solutions to avoid overfitting Train with more data that is by augmenting achieve this. Then early stopping means stopping the training process before the learner passes that point and Cross validation refers Partition the original training data set into k equal subsets. Each subset is called a fold.

2.13 Literature Review

Computer vision, deep learning, and object recognition in particular, has made tremendous advances in the past few years. Deep neural networks have recently been successfully applied in many diverse domains as examples of end to end learning. Neural networks provide a mapping between an inputs such as an image of a diseased plant to an output such as a crop disease pair. Various type of efforts have been developed to prevent crop loss due to diseases. Traditional approaches of widespread application of pesticides have in the past decade increasingly been supplemented by integrated pest management (IPM) approaches by author Ehler [9]. Before decades of time disease identification has been supported by agricultural extension organizations or institutions, like as local plant clinics. In more recent times, leveraging the increasing Internet penetration worldwide, such efforts have additionally been supported by providing information for online disease diagnosis. Even more recently, tools based on mobile phones have proliferated, taking advantage of the historically unparalleled rapid uptake of mobile phone technology in widespread of the world .Various types of images are using mobile phone, which are state by Geneva, International Telecommunication Union held on ICT Facts and Figures the World in 2015, can be utilized, and even nuances detection, classification, differences and brainy jobs [10]. Food security threatened by a number of factors including climate change [11], the decline in pollinators [12], plant diseases [13] and others. Identification of rice diseases using deep convolutional neural networks and pooling, resulted accuracy was 95.48% proposed by the authors Yang Lu et al [14]. The experiment evaluated with three different type of pooling methods namely max pooling, mean pooling, and stochastic pooling. Softmax regression learning algorithm can solve multi-classification problem they finally conclude that the Stochastic pooling method produces more accuracy 95.48% than others method. Limitation of these process does not answered how many layers and how many neurons are optimal at last. Process need high quality rice disease samples, faster and optimizing algorithm. Authors Jihen Amara et al [15] proposed A Deep Learning-based Approach for Banana Leaf Diseases Banana sigatoka and Banana speckle Classification. Deep learning models were trained under certain challenging conditions e.g. comprise of illumination, complex background, different images resolution, size and orientation effectively demonstrate the accuracy of this approach and the very less computational efforts required. Limitation of this paper does not confirm exact accuracy percentile and timing effort. Author Hughes et al [16] demonstrate the technical feasibility using a deep learning approach utilizing 54,306 images of 14 crop species with 26 diseases or healthy made openly available through the project PlantVillage, the limitation here is complex in feature. Computer vision, and object recognition in particular, has made tremendous advances in the past few years. The PASCAL VOC Challenge e. g PASCAL VOC project provides standardized image data sets for object class recognition. Also provides a common set of tools for accessing the data sets and annotations. Enables evaluation and comparison of different methods. Ran challenges evaluating performance on object class recognition from 2005-2012, now finished shown by Everingham et al [17] and more. And more recently the Large Scale Visual Recognition Challenge ILSVRC [18] Russakovsky et al based on the ImageNet dataset have been widely used as benchmarks for numerous visualization-related problems in computer vision, including object classification. A method that identifies

whether a rice leaf is healthy or infected accuracy of 99.83% shown by Usama Mokhtar et al [19]. The input image was first pre-processed by removing the background and, the noise present was eliminated by erosion technique. Gray Level Co-occurrence Matrix (GLCM) was given texture feature extraction from the enhanced image. Support Vector Machine (SVM) classifier was trained using different kernel functions, and the performance has been evaluated using N-fold cross-validation technique. Though accuracy high, it is not sufficient enough to predict or differentiate between healthy. Also, the type of disease was not identified. Authors Patil and Khirade [20] used various segmentation, feature extraction and classification techniques that identify, classify and detect the type of diseases using the diseased image. The leaf images pre-process either smoothing or enhancing applying histogram equalization. To confirm the affected area, various segmentation techniques like K-Means clustering have been proposed. The infected region were then extracted from the segmented region and calculated using GLCM. After feature extraction, the diseases can be detected with the help of Artificial Neural Networks (ANN) or Back Propagation Neural Networks. Drawback of segmenting the image using K-Means clustering is that the process proposed was semi-automated as the user has to explicitly select the cluster which contains the infection. Another disease detection approach Malus Domestic by image process by Bashir and Sharma [21]. Texture segmentation used algorithm names co-occurrence matrix and color segmentation used algorithm affect the quality of a system which understands image. K-means clustering has been used to partition the leaf image into four clusters in which one or more clusters contain the disease in case when the leaf shows the symptoms that has been infected by more than one disease. Limitations of these technique is K-means is semi-automated technique causes time consuming. Multi-Task Learning for Food Identification and Analysis with Deep Convolutional Neural Networks by Zhang et al. [22]. Xiachufang a chines cooking website images are the data, where cook image, cook method are provided, they clean data as data are user provided, using SVM trained with DCNN, label food category and merge data name for unity. SVM classifier trained by feature vector, achieves good classification results. After DCNN fully-connected layers are calculated by an activation function fact. Most of the activation functions are Rectified Linear Units (ReLU). SVM is not fully automated process so this time consuming process. In addition to data security issue will be raised as data provided by user. Microscope and DNA sequencing-based methods are effective to identify different kinds of diseases. Even though many of the farmers around the world do not have access to these diagnostics tools, the vast majority of them possesses a cell phone. In fact, the Ericsson company forecasts that mobile subscriptions will reach 9.3 billion in 2019 and 5.6 billion of these will be smartphone subscriptions shown by Interim report [23], phone utilize tool that helps in diagnosing crop diseases based on capturing and analyzing automatically a picture of a plant leaf is a promising solution by author Oerke [24]. CNNs are known for their robustness toward low variation in inputs, they require low preprocessing for their execution. They are also able to extract appropriate features while simultaneously performing discrimination correspondingly Deng, Arel et al [25] [26]. More specifically, authors Jihen Amara et al [27] in their paper implement to make use of the LeNet, LeCun et al [28] architecture for the convolution neural network. Apart these several LeNet efficient architecture utilized number of layer Yann LeCun et al [29] and Rizwan [30] or not Prajwala TM et al [31] or based on fixed layer Yann LeCun et al [32]. In gradient based learning problem, the more relevant measure is the error rate of the system. The error rate is estimated by measuring accuracy. The set of samples, disjoint from training set called test set, about these theatrical and practical job accordingly has shown by Sompolinsky and Seung, Vladimir Vapnik et al, Corinna Cortes et al, [33] [34] [35]. The difference between the expected error rate on the test E_{test} and the error rate on train set E_{train} decrease with number of train sample approximately as $E_{\text{test}} - E_{\text{train}} = k(h/O)\text{pow}(a)$. P number of train sample, h measure capacity either complexity

of machine between .5 and .1 and k is constant. The expected error rate shown by Vapnik and W. H. Press et al [36] [37] . If capacity increase, lowest E_{test} most algorithm try to minimize E_{train} where the variance is known as structural risk minimization.

2.14 Conclusion

This chapter revised practical and theoretical concepts of disease detection and classification method. In addition to the limitations and difficulties of classification and detection method are explained. Brief analysis of two potato disease problems are recognized. The acknowledgment lagging met when a trained plant pathologist or a farmer can diagnose disease. With fairly high confidence by looking at an image of symptomatic tissue pictorial. Such visual diagnosis does not scale well, perhaps; one person cannot efficiently gauge every herb leaf in a large area. Even if each plant could be clearly snapped. Disease shadowing applications can assistance from robust computer vision models that detect or classify objects in images. Trained using deep learning, where CNNs are a class of deep learning models that are currently state of the art in many computer vision tasks, including object classification and detection. Part of this success lies in the ability of a CNN to perform feature extraction along with label.

CHAPTER 3

METHODOLOGY

3.1 Introduction

In deep learning, CNN is a class of deep neural networks, most commonly applied to analyzing visual imagery also known as space invariant or shift invariant artificial neural networks (SIANN). CNNs are regularized versions of multilayer perceptron's usually mean fully connected networks (FCN). In FCN each neuron in one layer is connected to all neurons in the next layer which may makes them prone to overfitting data. In this chapter apply the methodology of convolutional neural network with need prerequisite to detect potato blight disease detection. In the previous chapter shows and discuss different approach of disease detection. Among them the easiest way disease detect could be applied with appropriate method.

3.2 Research Design

In this section, the basic steps for plant disease detection and Classification using image processing are shown. The thesis design is experimental observation analyzing light changing in the classification model. Basic steps for disease detection of the thesis is draw Figure 3.1.



Figure 3.1: Basic steps for disease detection.

- a). **Load images:** Dataset are loaded into jupyter notebook by python coding with importing uses libraries.
- b). **Process Images:** Then dataset are resize, greyscaling to run speed up. Process with segmentation, edge detection and softing.
- c). **Analyze model:** To apply convolutional neural network (CNN) any architecture make ready datas in step a,b and off course ensure balance datasets. Then the model created are save in directory and analyze with the tensorboard for better analyzing.and save in directory.
- d). **Optimizing Model:** Save model are optimized also with tensorboard and better decision making aproach are here by train model.
- e). **Use Train Model:** For this need testing data by get googling or capture image. The data is train and test with save model.
- f). **Classification:** After train and test the data with created model show the disease classification.

3.3 Sampling

Symptoms of late blight in the field are small, light to dark green, circular to irregular shaped water soaked spots (Figure 1.1). These lesions usually appear first on the lower leaves. Lesions often begin to develop near the leaf tips or edges, where dew is retained the longest. During cool, moist weather, these lesions expand rapidly into large, dark brown or black lesions, often appearing greasy (Figure

Effected late blight symptoms over stem and leaf shows Fig 2. During cool, moist weather, these lesions expand rapidly into large, dark brown or black lesions, often appearing greasy.



(a) Late blight effected stem (b) Late blight effected leaves

Figure. 3.2. Effected late blight symptoms on stem and leaf

A long period of growing, one or two spot severely damaged. Underside of leaves blanketed like flour. Consequently quickly get spoiled both leaves and stems Fig. 2(b) and 2(a).

Leaf lesions also frequently are surrounded by a yellow chlorotic halo. The lesions are not limited by leaf veins, and as new infections occur and existing infections coalesce, entire leaves can become blighted and killed within just a few days. The lesions also may be present on petioles and stems of the plant. For sample potato leaves are used. Effected potato crops in early blight diseases is shown in Fig. 3.



(a) Early blight effected stem (b) Early blight effected leaves

Figure. 3.3 Effected early blight symptom on stem and leaf

Symptoms of early blight appear as small, circular or irregular, dark-brown to black spots on leaves Fig 3.3(a). Innumerable spot spread across the leaves cause to dry out Fig 3.3(b).

Symptoms of early blight appear as small, circular or irregular, dark-brown to black spots on the older leaves. These spots enlarge up to 3/8 inch in diameter and gradually may become angular-shaped. Disease image leaves are collected from the district of Noakhali Sonapur -314, Sonaimuri- 3827 belongs two villages Papua and Shimulia and Dhaka

Banasree in potato yard and garden. Approximately three thousand image are collected 2,100 images are collected, for train process use 1960 data, rest of data are separated for test purpose. Affected potato of early blight and late blight potatoes are twenty.



| (a) Late blight (b) Early blight

Figure 3.4: Affected potatoes early and late blight.

Potato blight disease crops are collected from the district of Noakhali Sonapur -3814, Sonaimuri- 3827 survey three different potato garden. Acquisition are manually place by single potato and capture with smart devices. Device having default measurements and resolutions are and have change device to device. For capturing digital devices are Iphone6, Alcatel, Xiami both Android and ios platform. Effected potato crops collection are fifty in number. To feed the world agricultural crops must be cooprated through production and should be protected from from disease for giving assurance of high rate of production.

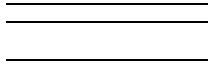
3.4 Instrument

- a. Digital camera or Smart Phone: This is need for diseased image capture, HD resolution provide images details clearly, will use for train and test.
- b. Anaconda software: Prepare Image as input, required input will be collected from jupyter notebook web application and google colaboratory.
- c. Bulkresizephotos, RIOT, MATLAB software or site: To Image resize and further effect python coding and these software needed.
- d. WPS server: Data will be uploaded into an online data storage and make directory which ensure security and prepare each disease images as a directory or class.
- e. Python language and libraries: To Train images programming and development will be done with API of python language and Tensorflow package along high level API of Keras.
- f. GPU, Tensor board, Computer and smart phone: Experiment and Accuracy Testing will be done with Dell Inspiron 3567 7th Gen Core i3 Laptop with Genuine Win 10 and keras API in data storage and cloud storage with default drive.

g. Online site and smart devices: Classification result and detect suggestion will be done online org <https://playground.tensorflow.org> and Jupyter Notebook, colab ensure disease suggestion will be conducted using them.

3.5 Steps for Proposed Method

The proposed scheme includes these important steps namely Data Acquisition, Data pre-processing and Classification etc. In data acquisition contain steps are Raw Leaves collection, process image, and store image for next step. The proposed methodology full process here,



Classification

| Figure 3.5: Proposed Methodology

3.5.1 Raw Leaves collection

Collected or survey leaves are collected from different yard garden etc. Figure 3.4.1 shows the image of agriculture field survey with different locations where raw leaves are collected.



Figure 3.6: Field survey of potato leaves.

Potato blight disease leaves or raw leaves are collected from the district of Noakhali Sonapur -3814, Sonaimuri- 3827 belongs two villages Papua and Shimulia containing three different potato garden. Also Dhaka Banasree in potato yard and garden. Acquisition are manually place by single leaves and capture with smart devices. Device having default dimensions and resolutions are and have variance device to device. Devices are Iphone6, Alcatel, Xiomi Android ios, and digital devices. Around 2,100 leaves are store from collected images. 980+980 image are prepared for process, 1960 rest of data are separated for test purpose. All the collected data firstly need to process for being same feature.

3.5. 2 Image Process

As previous section disease images are separately organized and exquisite for further process due to same capacity, feature, channel, labeling corresponding devices etc. Fig 8 shows potato early and late blight affected potato image and deeper severity label corresponding disease.

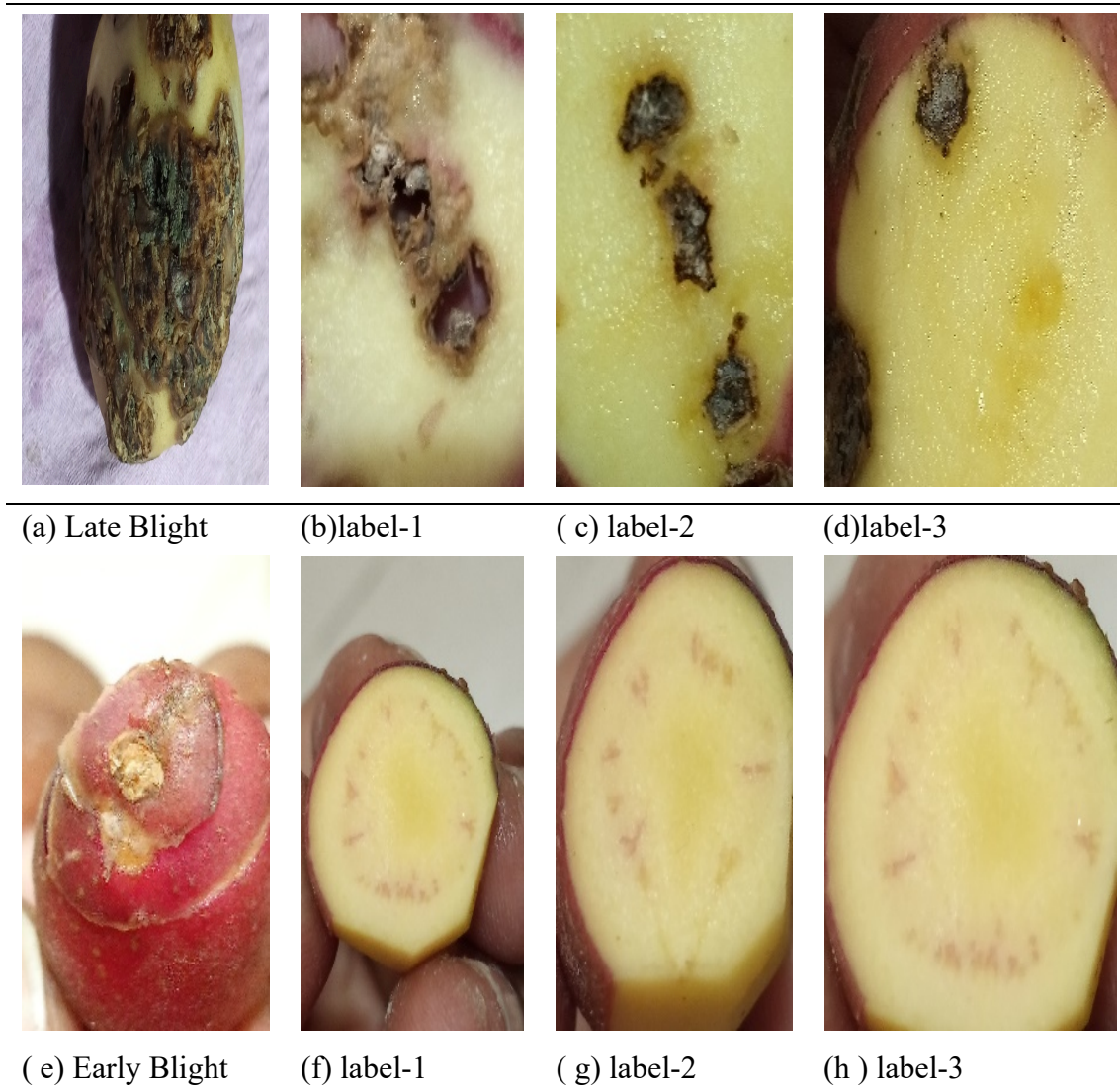


Figure 3.7 Affected Late Blight and Early Blight potato with deeper severity.

Late blight disease potato rotten prior dry out Fig 8(b) (c) (d) where early blight potato spot circular get larger by splicing Fig 8 (f) (g) (h). Late blight and early blight severity visually leaves and affected on potatoes by slicing shows on Figure 3.7, also indicate second row early blight symptoms and affected level in potatoes, early blight problem occur as circular lesion and quickly become severe if any action not taken. Early blight circular spotted in potato with potato leaves lesion growth, these symptoms shows early growth time of potato plant. And first row shows late blight symptoms and affected severity in potatoes. Late blight disease occur late age of plant and dry out affected leave spot besides potato get rotten. Late blight appeared affected leaves stem and below surface of leaves black spot with whites appeared. The disease potato leaves images have been consists of approximate around three thousand images belonging to two different class's early blight and late blight diseased image in storage. [38]. The dataset includes images of all major kinds of leaf diseases that could affect the potato crop. By default each of the images belongs to the RGB color space and are JPG format configuration of each capture digital device. CNN does work with same capacity, size, feature, color and capacity. With the help of online web application bulk resize and windows default editor edited images to same capacity, size 148 x 148 and make sure are same image format. All the images needed to be noise free and for data preprocess for saving in the cloud storage as sample.

3.5. 3 Data Storage

In the cloud, sample are stored separately in two class for two purpose. One reason for train data with one class and other reason test model with the other class sample. Data are stored in cloud storage for the further processing and cloud GPU support. Require as convolutional neural network maintain same capacity, size, shape so data stored classes must be same in capacity that is for train data early blight 980 image and late blight 980. Rest of sample are belongs to test class. To make them same as primarily dimension prior implemented data. All the images are color of three channel.

3.5.4 Data preprocess

This dataset consisted of images with minimal dimension. Author Lim discuss digital image and signal process [39] Now look at one of the images in dataset to see what are working with. Plot the first image in dataset and check its size using the shape function by import matplotlib library. The images in the dataset had resized to 148×148 resolution in Image Process section, convert them into image array. Read Image as one channel format. Further speed up the training process to make model training computationally manageable. Resize array matrix steps.

- Resize Image Array (Computation)

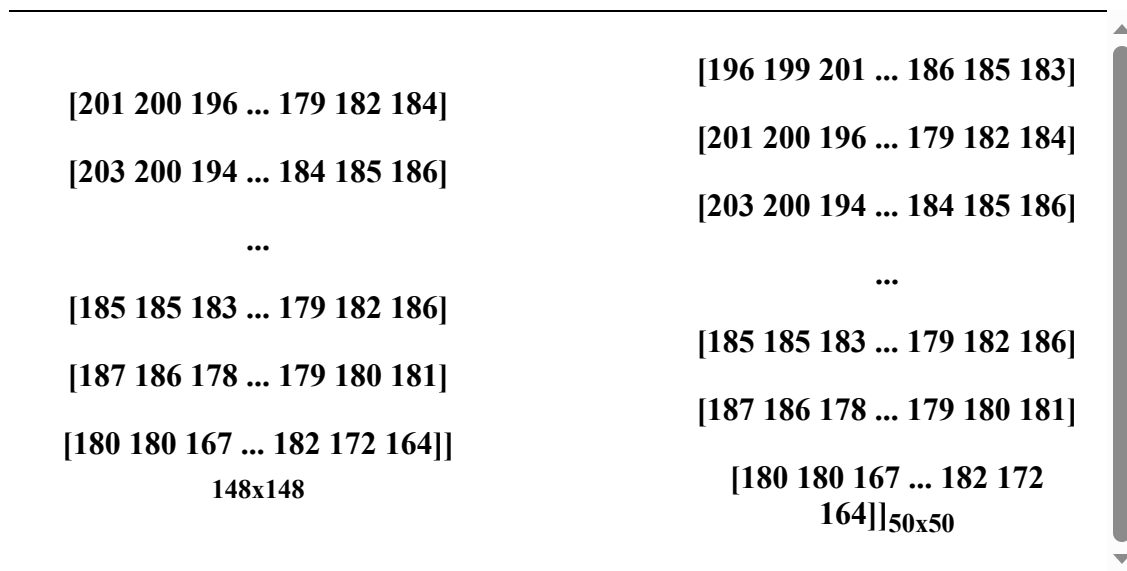


Figure 3.8: Resize Image Array

3.5. 5 Train Data

In this steps data are manually classes, so now in implemented with python, need to train data. To create training data and all that, but, first, set aside some images for final testing which already done in section Data Storage, Just manually create a directory called Testing and then create 2 directories inside of there, one for Early Blight and one for Late Blight. Make sure all the images are not copy of other class. Testing class images are use t for final tests. Now, want to begin building training data. Give index to run data as directory list of categorical or indexing. For each class contains nineteen hundred eighty images. For the first train of dataset took high speed internet connection provided GPU support.

100% ██████████ 981/981 [07:25<00:00, 1.86it/s]

100% ██████████ 981/981 [06:55<00:00, 2.66it/s]

Figure 3.9: Train Data Time Speed

Almost 1960 samples are trained on Fig 3.9. One thing want to do make sure train data is balanced. In the case of this dataset, the dataset started off as being balanced. By balanced, mean there are the same number of samples for each class such as same number of early blight and late blight. If not balanced, either want to pass the class weights to the model, so that measure error appropriately, or balance samples by trimming the larger set to be the same size as the smaller set. If not balance, the model will initially learn that the best thing to do is predict only one class, whichever is the most common. Then, it will often get stuck here. In this case though, this data is already balanced, so that's easy enough. Also, if the dataset that is too large to fit into ram, need to batch-load in data. There are many ways to do this, some outside of Tensorflow and some built in. Certainly data for now is balanced, in this section mainly trying to cover how data should look, be shaped, and fed into the models. Need shuffle the data. Right now data is just all early blight, then all late blight. This will usually wind up causing trouble too, as, initially, the classifier will learn to just predict early blight always. Then it will shift to oh, just predict all late blight. Going back and forth like this is no good either. By import random library can shuffle the data as training data is a list, meaning it's mutable, so it's nicely shuffled [40]. By iterating over a few of the initial samples and printing out the class can confirm mix data. Classes are nicely mixed in. Build model and save this data, so that don't need to keep calculating it every time, just want to play with the neural network model. Got dataset, ready to cover convolutional neural networks and implement one with prepared data for classification.

3.5. 6 Deploy CNN and Analyze model

The Convolutional Neural Network gained huge popularity through its use with image data, and is currently the state of the art for detecting what an image is [41] [42] and classification of large classes [43]. The basic CNN structure [44] is as follows: Convolution, Pooling, Convolution, Pooling, Fully Connected Layer, Output Convolution is taking the original data, and creating feature maps from it. Pooling is down-sampling, most often in the form of "max-pooling," where select a region, and then take the maximum value in that region, and that becomes the new value for the entire region. Fully Connected Layers are typical neural networks, where all nodes are "fully connected." The convolutional layers are not fully connected like a traditional neural network. Each convolution and pooling step is a hidden layer. After this, we have a fully connected layer, followed by the output layer. The fully connected layer is typical neural network (multilayer perceptron type of layer, and same with the output layer.

- Model Prerequisite

Prior convolutional neural network there are some prerequisite. First convert this dataset to training data. As a few issues right out of the gate. The largest issue is not all of these images are the same size. While eventually have variable sized layers in neural networks, this is not the most basic thing to achieve. Want to reshape things, so every image has the

same dimensions. Next, may or may not want to keep color. After training data need to randomize for clarification. As provided indexing this shown as 0 or 1 as the validation result. For feature say xx and label say yy of each image need to reshape data.

Reshape Image array:

```
Data = image array * (-1, 50, 50, 1)
```

Or

```
Data = image array * (-1, 70, 70, 1)
```

Figure 3.11: Image pre-process Implement-View.

Need to reshape dataset inputs to the shape that model expects when train the model. The last number is 1, which signifies that the images are greyscale. The process of input variables tends to speed up the training process through improvement of the numerical condition of the optimization problem. Several default values involved in initialization and termination are appropriate. Image segmentation [45] providing a label to every pixel in an image so pixels with the same label share certain features. In consequence's region of interest (ROIs) find out the doubtful mass candidates from the ROI by using segmentation [46]. But proposed approach get feature and label by reshaping. Rather their feature and label are normalized with per image each pixel divided 255 as the one channel image pixel value range 0 to 255. The result of each each pixel will get 0 or 1 to speed the computation more efficiently [25] [26] both of these feature extraction need more computation but current makes more simply.

- Model Analyze

Computers see images using pixels. Pixels in images are usually related. Normalization applied with image array for single value zero or one. Sequential [47] [48] model are utilized here with Adam optimizer along CNN. Convolutional two layer are use, each layer Conv filter 256, MaxPooling use 2x2, advance activation ReLU are used. Convolutions use this to help identify images. A convolution multiplies a matrix of pixels with a kernel that is filter matrix and sums up the multiplication values. Then the convolution slides over to the next pixel and repeats the same process until all the image pixels have been covered. Assume each square is a pixel. For the convolution step, going to take a certain window, and find features within that window. That window's features are just a single pixel-sized feature in a new feature map, but will have multiple layers of feature maps in reality. Next slide that window over and continue the process. There will be some overlap, can determine how much wanted, just do not want to be skipping any pixels, of course. Now continue this process until have covered the entire image, and then will have a feature map. Typically the feature map is just more pixel values, just a very simplified one.

- Conv2D Filters:

Image

K kernel applied Each Kernel conv with image Feature Map

With input image

Figure 3.11: Conv2D parameter filters

Conv2D determines the number of kernels to convolve with the leaves image and produces a 2D activation map or feature map each time.

- MaxPooling: Kernel size (2, 2):

5	4
1	2
2	1
◀ ◯ ▶	
2	2
2	2
◀ ◯ ▶	
5	5
2	3

Image

Figure 3.12: MaxPooling

This time data pixels are scan with kernel size and select highest pixel by given kernel. Kernel size can be variance either 1x1, 2x2, 3x3, 4x4, 5x5. Here image pixel 3x3, kernel size 2, 2 produce MaxPooling result 2, 2.

- I. ReLU Activations can be used through an Activation layer. This returns neuron fire or not any feature. Activation is the activation function for the layer. The activation function for first 2 layers is the ReLU, or Rectified Linear Activation. This activation function has been proven to work well in neural networks.
- II. Dropout: due to disabling neurons, some of information about each sample is get loss. And the subsequent layers attempt to generate the data result basing on incomplete representations.

The exponential linear activation:

X if $x > 0$ and $\alpha * (\exp(x) - 1)$ if $x < 0$.

This is a layer of CNN then another one used for this CNN deployment. After that 3D vector need to Flatten to convert 1D vector.

- III. **Dense Layer:** Dense means input parameters, with neurons in the FIRST hidden layer.

IV. **Compilation** the model

Compiling the model takes three parameters: optimizer, loss and metrics [47].

Prior train the model, learning process need to configure via the compile [49] method. It receives three arguments:

An Optimizer. This could be the string identifier of an existing optimizer. The optimizer controls the learning rate. Have Used Adam as optimizer in this work. Adam is generally a good optimizer to use for many cases. The Adam optimizer adjusts the learning rate throughout training.

A Loss function. This is the objective by which model will try to minimize. It can be the string identifier of an existing loss function such as categorical_crossentropy or other. This is the most common choice for classification. A lower score indicates that the model is performing better. The learning rate determines how fast the optimal weights for the model are calculated. A smaller learning rate may lead to more accurate weights that is up to a certain point, but the time it takes to compute the weights will be longer.

A list of metrics. For any classification problem need to set this to metrics= ['accuracy']. To make things even easier to interpret, use the 'accuracy' metric to see the accuracy score on the validation set when train the model.

3.5.7 Optimize and Build Model

To optimize model check with conv layer 32, 64, 128 etc. with window 3x3. So which conv layer accurate check with 64 here.

- Optimize model

Update with CNN model with smaller convolutional filter each layer 64, Activation layer of ReLU and MaxPooling with window size 2.

Conv2D (64)

ReLU Activation

MaxPooling2D (size (2x2))

Conv2D (64)

ReLU Activation

MaxPooling2D (size (2x2))

.....

MLP

Figure 3.13: A small design of CNN

- Build Model

To finalize the model we use Tensorboard and run with three different layer 32, 64,128 sizes, this running process need GPU, conv layer and 3 different dense layer. Layers size are sequential with power of 2. After run select 10 best layers.

Among them run three each layer to find better accuracy. Finalize the model.

1-conv-64-nodes-0-dense-1583741819

2-conv-256-nodes-0-dense-1583741819

3-conv-64-nodes-0-dense-1583741819

conv	nodes	dense
1	64	0
2	256	0
3	64	0

Table 3.1: Three best layers table.

In this step model with the best layer 1-conv-64-nodes-0-dense is gives accuracy 98% and validation accuracy 96%. Now this time model is created for classification.

3.5.8 Classification

Convolutional neural networks (CNN) can be used for the creation of a computational model that works on the unstructured image inputs and converts them to corresponding classification output labels. For the purpose of potato leaf disease detection, we have experimented with several standard deep learning architectures like AlexNet [50], GoogleNet [51] and the best results could be seen with the use of a variation of the LeNet architecture [52] [53]. LeNet is a simple CNN model that consists of convolutional, activation, pooling and fully connected layers. The architecture used for the classification of the potato leaf diseases is a variation of the LeNet model [54]. Current CNN use sequential model adam optimizer with default params. Convolutional layer applies convolution operation for features extraction. With the increase in depth, the complexity of the extracted features increases. The size of the filter is fixed to 5×5 whereas number of filters is increased progressively as they move from one block to another. The parameter table that are used during practical use view on Table 3.2.

Name	Keyword Params
Adam Optimizer	Optimization algorithm, Learning Rate, Momentum, Weight Decay, beta_1, beta_2, amsgrad.
Model Fit	Batch Size, Epoch iteration
CNN layer	Conv layer, Activation function, Pooling
Compile	binary_crossentropy, Adam, metrics.
Epochs	20

Table 3.2: Model Parameter table

The number of filters like three in the first convolutional block while it is increased to twenty in the second in this step just use conv, dense and layer sizes as best layer requirement. This not need increase in the number of filters is necessary to compensate for the reduction in the size of the feature maps caused by the use of pooling layers in each of the blocks. CNN block used conv2D filter 64 then 256, kernel size 3by 3 and

input shape same as input image. For model fit function parameter are fed namely Optimization algorithm, Learning Rate, Momentum, Weight Decay, beta_1, beta_2, amsgrad. Batch size 32 and epoch iterated for twenty times. Compile function used params are binary_crossentropy, Adam, metrics.

3.6 Conclusion

This chapter brush up practical models of disease detection and classification. In addition to the troubles of classification and detection are elucidated. Brief analysis of two potato disease problems are recognized. Part of this implementation lies in the ability of a CNN to perform feature extraction, tuning parameter optimize the model. The feature maps are also same size in order to domain the size of the image after the use of the convolution operation. The max pooling layer 2×2 , perform the feature maps, speeding up the training process, and making the model less variant. Advance ReLU activation layer is used in each of the blocks for the introduction of non-linearity to minor changes in input.

CHAPTER 4

RESULT AND DISCUSSION

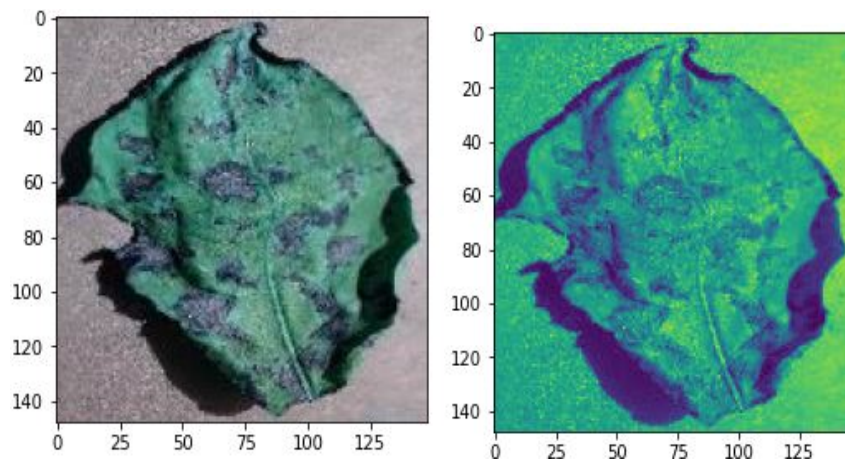
4.1 Introduction

This chapter shows result which are applied in the methodology chapter all the steps result and details about the output. The work main steps are pre-process, train model, analyze model, optimize model and then classify with distinct ample. In the last of this chapter could be compare with the proposed methodology according result.

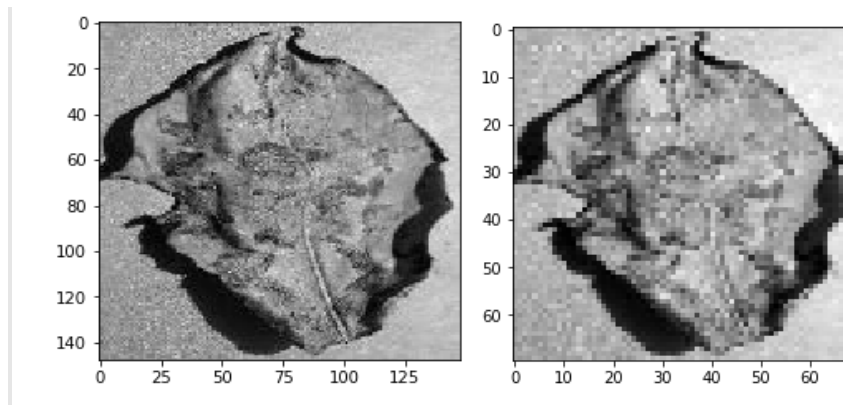
4.2 Pre-process Output

For resize check with the output either with 50×50 or 60×60 or 70×70 . Prepared sample check a image with shape outputted as (148, 148, 3) where height 148, weight 148 and color channel is 3 because it's RGB. Definitely don't want the images that big, but also various images are different shapes. Check with 100×100 would better but 70×70 is much better for the dataset.

Resize Image Array in graphical show on Figure 4.1:



(a) 148X148 RGB (b) 148X148 gray



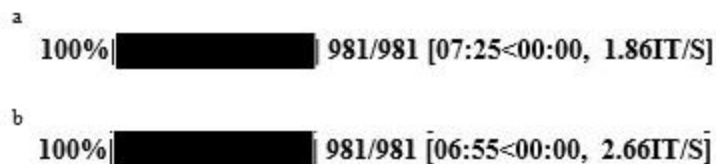
(c) 70X70 gray (d) 50X50 gray

Figure 4.1: Image channel modification.

Figure 4.1 shows top left 4.1(a) is 148x148 RGB that is 3 channel, (b) is 148x148 with color map gray, (c) is 70x70 with color map gray and read grayscale on one channel, (d) 50x50 with color map gray and read grayscale in one channel.

4.3 Train Data Output

In this steps data with early blight images class train speed 1.86IT/S the other class late blight image train speed 2.66IT/S. GPU alter memory to accelerate the creation of images in a frame buffer intended for output to a display device here display device laptop.



.Figure 4.2: Train data with speed.

Without GPU compile these image is so time consuming and lengthy process. Modern GPUs are very efficient at manipulating computer graphics and image processing. In a personal computer, a GPU can be present on a video card or embedded on the motherboard.

4.4 First Deploy CNN

MaxPooling and ReLU activation are mutually represent a hidden layer. Uses CNN compile with two of such hidden layer. After just three epochs, 78% validation accuracy. If keep going, probably do even better, but should probably discuss how to improve validation accuracy. To help with this, use Tensorboard, which comes with Tensorflow and it helps to visualize models as they are trained. Proposed CNN model is mention on Figure 4.3.

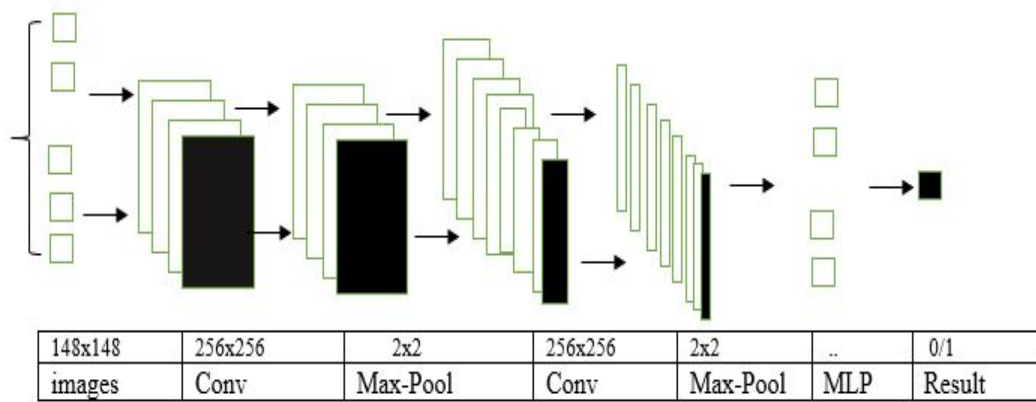


Figure 4.3: Deployed CNN overview

Conv2D parameter is the numbers of filters that convolutional layers will learn from. It is an integer value and also determines the number of output filters in the convolution. Figure 4.3 learning a total of 256 filters and then use 2 by 2 Max Pooling to reduce the spatial dimensions of the output volume with two hidden layer. ReLU activation function with dense layer prior use flatten function to convert one dimension. Softmax dimension into one dimension format

for three epochs.

So first Running CNN model shows on Figure 4.3:

```
Model: "sequential_37"

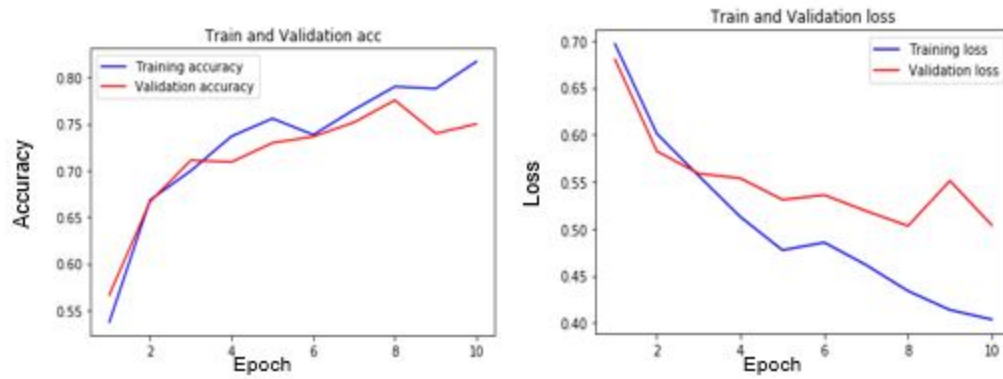
Layer (type)                 Output Shape              Param #
-----
conv2d_74 (Conv2D)           (None, 48, 48, 128)      1280
activation_111 (Activation)   (None, 48, 48, 128)      0
max_pooling2d_74 (MaxPooling (None, 24, 24, 128)      0
conv2d_75 (Conv2D)           (None, 22, 22, 256)      295168
activation_112 (Activation)   (None, 22, 22, 256)      0
max_pooling2d_75 (MaxPooling (None, 11, 11, 256)      0
flatten_37 (Flatten)         (None, 30976)             0
dense_74 (Dense)              (None, 64)                1982528
dense_75 (Dense)              (None, 1)                  65
activation_113 (Activation)   (None, 1)                  0
Total params: 2,279,041
Trainable params: 2,279,041
Non-trainable params: 0

Train on 1372 samples, validate on 588 samples
Epoch 1/3
1372/1372 - 1s - loss: 0.7080 - accuracy: 0.5751 - val_loss: 0.6352 - val_accuracy: 0.6207
Epoch 2/3
1372/1372 - 1s - loss: 0.5180 - accuracy: 0.7194 - val_loss: 0.5569 - val_accuracy: 0.7109
Epoch 3/3
1372/1372 - 1s - loss: 0.4683 - accuracy: 0.7522 - val_loss: 0.4869 - val_accuracy: 0.7857
```

Figure 4.4: First CNN Compilation.

From the Figure 4.3 run output accuracy is 78% accuracy as this found rerun the CNN deployment and Train on 1372 samples, validate on 588 samples. Separate the data 30% for validation and 70% for accuracy testing. So need further analyze. So the model does not create here to create the final model. Training and testing purpose splitting result are plot on graph.

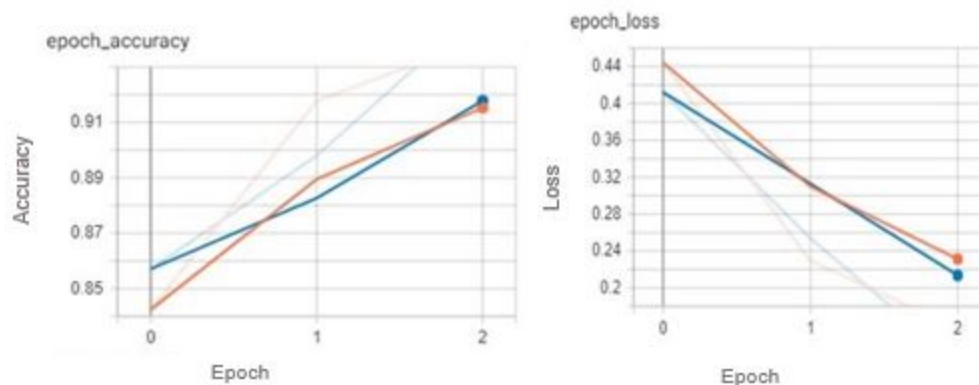
Figure 4.4 provide the both train and test loss and accuracy accordingly epochs.



(a) 70% Accuracy and Validation of 30% (b) 70% Loss accuracy and Validation is 30%.

Figure 4.5: Data Accuracy and Loss graph

Figure 4.5 (a) horizontal axis for epoch and vertical for accuracy in percent unit. Color as blue for training accuracy where data is 70% and red for validation accuracy where data amount is 30%. graph 4.5 (a) shows validation accuracy get almost accurate with training accuracy that means training accuracy above 80% after three epochs. Graph 4.5 (b) horizontal axis for epoch and vertical for Loss in percent unit. Training loss blue and red for validation loss which shows 75%. The more minimize loss the data get validate. Training loss is 40% and validation loss nearly 50% after three epochs. To increase validation accuracy, Tensorboard can help with this, which comes with Tensorflow and visualize models as they trained [55]. From figure 4.3 first model run output is 78% accuracy, CNN deployment and Train on 1372 samples, validate on 588 samples. Separated the data 30% for validation and 70% for accuracy testing. So need further analyze with Tensorboard. Tensorboard can be used directly within notebook experiences such as Colab and Jupyter. This can be helpful for sharing results, integrating Tensorboard into existing workflows, and using Tensorboard without installing anything locally. In this research load first deployed CNN model in Tensorboard with a few command. Both of train and test loss and accuracy accordingly epochs are shows on Figure 4.6.



(a) Accuracy Graph (b) Loss graph

Figure 4.6: Epoch Accuracy and Loss with the Tensor Board.

Dashboards use for graphs feature figure 4.6(a) show horizontal axis for epoch and vertical for accuracy in percent unit. The graph shows blue progress line for model training accuracy 93% and red for validation accuracy 92%. 4.6(b) show horizontal axis for epoch and vertical for loss in percent unit. Graph shows blue progress line for model training loss 0.2 and red for validation loss slight above .24. The same Tensorboard backend is reused by issuing command, it monitor model in progress. If a different logs directory was chosen, a new instance of Tensorboard would be opened. A logs class is create in the cloud storage to load and run the model with Tensorboard. Ports are managed automatically. Start training a new model and watch Tensorboard update automatically every 30 seconds or refresh it with the button in the Tensorboard.

4.5 Analyze and Optimize model

Tensorboard is a handy application that allows you to view aspects of your model, or models, in your browser. The way that use Tensorboard with Keras is via a Keras callback. There are actually quite a few Keras callbacks. Definitely check the others out Keras Callbacks [56].

4.5.1 Analyze with different size model

Model Checkpoint is another useful one. For now, however, focused on the Tensorboard callback. Callbacks is a list. And could pass other callbacks into this list as well and enable visualizations for Tensorboard. Inherits from Callback, Tensorboard is a visualization tool provided with Tensorflow. This callback logs events for Tensorboard, including, Metrics summary plots, Training graph visualization, Activation histograms, Sampled profiling. Change some things in the model. To begin, added an activation to dense layer. This layer repeat once more time to analyze train loss, accuracy and test loss, validation accuracy. Analyze model 64, 256 convolution and MaxPooling 3x3 filter. Arguments log_dir the path of the directory where to save the log files to be parsed by Tensorboard. For the implementation only log_dir is used. The path log_dir redirect to cloud storage to load model into Tensorboard. Thus visualize layers whose are best in performance epoch accuracy with validate. In Tensorboard Toggle button correspond to each layer, layers among all best ten are toggled by toggle button on the Tensorboard. A small model implementation summery Backend with keras on Figure 4.7.

Model: "sequential_46"

Layer (type)	Output Shape	Param #
conv2d_92 (Conv2D)	(None, 48, 48, 64)	640
activation_146 (Activation)	(None, 48, 48, 64)	0
max_pooling2d_92 (MaxPooling)	(None, 24, 24, 64)	0
conv2d_93 (Conv2D)	(None, 22, 22, 64)	36928
activation_147 (Activation)	(None, 22, 22, 64)	0
max_pooling2d_93 (MaxPooling)	(None, 11, 11, 64)	0
flatten_46 (Flatten)	(None, 7744)	0
dense_92 (Dense)	(None, 64)	495680
activation_148 (Activation)	(None, 64)	0
dense_93 (Dense)	(None, 1)	65
activation_149 (Activation)	(None, 1)	0

Figure 4.7: A small model implementation summery.

For the model type that used is Sequential. This is the easiest way to build a model in Keras. Sequential allows to build a model layer by layer dot add '.add ()' function to add layers to the small model. First 2 layers are Conv2D layers. These are convolution layers that will deal with input images, which are seen as two dimensional matrices. 64 in the first layer and 64 in the second layer are the number of nodes in each layer. This number can be adjusted to be higher or lower, depending on the size of the dataset. In this case, 64 work well. Kernel size is the size of the filter matrix for our convolution. So a kernel size of 3 means a 3x3 filter matrix. Activation is the activation function for the layer. The activation function being used for first 2 layers, is the ReLU, or Rectified Linear Activation. Since this activation function has been proven to work well in neural networks. In between the Conv2D layers and the dense layer, there is a 'Flatten' layer. Flatten serves as a connection between the convolution and dense layers. Dense is the layer type we will use in for our output layer. Dense is a standard layer type that is used in many cases for neural networks. Figure 4.8 shows a small model with val_accuracy 75%.

```

total params: 533,313
Trainable params: 533,313
Non-trainable params: 0

Train on 1372 samples, validate on 588 samples
Epoch 1/10
1372/1372 - 1s - loss: 0.6964 - accuracy: 0.5372 - val_loss: 0.6800 - val_accuracy: 0.5663
Epoch 2/10
1372/1372 - 0s - loss: 0.6011 - accuracy: 0.6684 - val_loss: 0.5824 - val_accuracy: 0.6667
Epoch 3/10
1372/1372 - 0s - loss: 0.5563 - accuracy: 0.6997 - val_loss: 0.5587 - val_accuracy: 0.7109
Epoch 4/10
1372/1372 - 0s - loss: 0.5125 - accuracy: 0.7369 - val_loss: 0.5536 - val_accuracy: 0.7092
Epoch 5/10
1372/1372 - 0s - loss: 0.4773 - accuracy: 0.7558 - val_loss: 0.5309 - val_accuracy: 0.7296
Epoch 6/10
1372/1372 - 0s - loss: 0.4855 - accuracy: 0.7383 - val_loss: 0.5359 - val_accuracy: 0.7364
Epoch 7/10
1372/1372 - 0s - loss: 0.4615 - accuracy: 0.7653 - val_loss: 0.5189 - val_accuracy: 0.7517
Epoch 8/10
1372/1372 - 0s - loss: 0.4339 - accuracy: 0.7901 - val_loss: 0.5031 - val_accuracy: 0.7755
Epoch 9/10
1372/1372 - 0s - loss: 0.4138 - accuracy: 0.7879 - val_loss: 0.5510 - val_accuracy: 0.7398
Epoch 10/10
1372/1372 - 0s - loss: 0.4037 - accuracy: 0.8171 - val_loss: 0.5039 - val_accuracy: 0.7500

```

Figure 4.8: Small model with val_accuracy

Figure 4.8 shows data in Validation Split as 70% and 30%. The small implementation result shows that loss 40% which less than test loss 50%, train accuracy is 81% which is greater than validation accuracy 75%. Total parameter is 533,313, Trainable params 533,313 and non-triable params 0. As validation accuracy is 75% that means should increase conv layer from 64.

4.5.2 Full model analyze

Exemplify might build a workflow to optimize model's architecture. To begin, A few things could do to the model figure 4.5. The most basic things to modify are layers and nodes per layer, such as 0, 1, or 2 dense layers. Even parameters will take some significant time like varying layer sizes, activation functions, learning rates, dropouts, and much more. In previous compilation 64 node-per-layer is working for 75% validation accuracy. So more optimize compile 128 and a 256, along with 64, list is [32, 64, 128] is try 64. 128 shows better results, so then tried [64, 128, 256] and so on. Change in certain parameters, may need to revisit older ones, so ensure that is add or just increase dropout, likely have a larger overall model in terms of layers or nodes per layer than before. A larger model overall might want a larger starting learning rate or a slower rate of decay for the learning rate. Small and few of run increasing or decreasing conv layer get this output.

Layer (type)	Output Shape	Param #
conv2d_173 (Conv2D)	(None, 48, 48, 128)	1280
activation_300 (Activation)	(None, 48, 48, 128)	0
max_pooling2d_173 (MaxPoolin	(None, 24, 24, 128)	0
conv2d_174 (Conv2D)	(None, 22, 22, 256)	295168
activation_301 (Activation)	(None, 22, 22, 256)	0
max_pooling2d_174 (MaxPoolin	(None, 11, 11, 256)	0
flatten_87 (Flatten)	(None, 30976)	0
dense_165 (Dense)	(None, 64)	1982528
activation_302 (Activation)	(None, 64)	0
dense_166 (Dense)	(None, 1)	65
activation_303 (Activation)	(None, 1)	0

Figure 4.9: Full update model implementation summery.

Compilation result of the full update model compilation of 70% and 30% loss, accuracy, val_loss, and val_accuracy shows on Figure 4.10:

```

Total params: 2,279,041
Trainable params: 2,279,041
Non-trainable params: 0

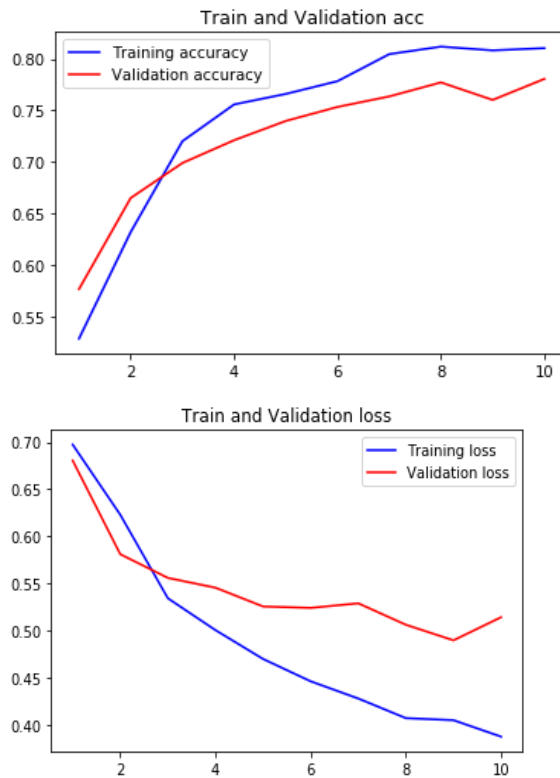
Train on 1372 samples, validate on 588 samples
Epoch 1/10
1372/1372 - 1s - loss: 0.6974 - accuracy: 0.5284 - val_loss: 0.6804 - val_accuracy: 0.5765
Epoch 2/10
1372/1372 - 1s - loss: 0.6228 - accuracy: 0.6319 - val_loss: 0.5809 - val_accuracy: 0.6650
Epoch 3/10
1372/1372 - 1s - loss: 0.5341 - accuracy: 0.7201 - val_loss: 0.5558 - val_accuracy: 0.6990
Epoch 4/10
1372/1372 - 1s - loss: 0.5003 - accuracy: 0.7558 - val_loss: 0.5454 - val_accuracy: 0.7211
Epoch 5/10
1372/1372 - 1s - loss: 0.4697 - accuracy: 0.7660 - val_loss: 0.5254 - val_accuracy: 0.7398
Epoch 6/10
1372/1372 - 1s - loss: 0.4460 - accuracy: 0.7784 - val_loss: 0.5239 - val_accuracy: 0.7534
Epoch 7/10
1372/1372 - 1s - loss: 0.4277 - accuracy: 0.8047 - val_loss: 0.5288 - val_accuracy: 0.7636
Epoch 8/10
1372/1372 - 1s - loss: 0.4068 - accuracy: 0.8120 - val_loss: 0.5060 - val_accuracy: 0.7772
Epoch 9/10
1372/1372 - 1s - loss: 0.4047 - accuracy: 0.8083 - val_loss: 0.4895 - val_accuracy: 0.7602
Epoch 10/10
1372/1372 - 1s - loss: 0.3872 - accuracy: 0.8105 - val_loss: 0.5140 - val_accuracy: 0.7806

```

Figure 4.10: Full update model compilation

The small implementation result shows that loss 38% which less than test loss 50% , train accuracy is 81% which is greater than validation accuracy 78%.This accuracy result is more validate than small model. Total parameter is 2,279,041,

Trainable params 2,279,041 and non-triable params 0. Figure 4.9 provide the both train and test loss and accuracy accordingly epochs.



(a) Train and validation accuracy (b) Train and validation loss

Figure: 4.11. Full model train and validation accuracy Plot.

Fig 4.11 (a) plot as blue for training accuracy where data is 70% and red for validation accuracy where data amount is 30%. In addition validation accuracy gets almost accurate with training accuracy that means training accuracy above 80% after iterating three epochs Fig 4.11(b) shows that blue color for training loss and red for validation loss which shows near 80%. As well as graphs, training loss is 40% and validation loss nearly 50% after three epochs.

4.6 Experiment Result

Selected data predicted with the save model. Work as a function every for selected data similar to predicted model optimization size capacity dimension to predict result as early blight disease or late blight disease. Experiment result shows fluctuation of the model summary with model compilation parameter such as loss, accuracy, validation accuracy and Validation_LOSS. All the compilation and optimization done with the help of keras API background upgrade Tensorflow in python. Tensorboard analyzer contains more than one parameter. Such image histogram, graph, epoch accuracy epoch loss etc. Toggle of Tensorboard function can toggle to visualize each selected layer performances. For CNN usually besides optimize and analyze sometime. Epoch number increase and compilation number increase can give best performance of the model.

3-conv-64-nodes-1-dense-1581577627

Train on 980 samples, validate on 980 samples

Epoch 1/20

980/980 -3s – loss: 0.7016 – accuracy:0.5204 - val_loss:0.6713 - val_accuracy:0.5898

Epoch 2/20

980/980 -1s – loss: 0.4640 – accuracy: 0.7776 - val_loss: 0.2191- val_accuracy: 0.9102

Epoch 3/20

980/980 -1s – loss: 0.1988 – accuracy: 0.9184- val_loss: 0.1848- val_accuracy: 0.9316

Epoch 4/20

980/980 -1s – loss: 0.1696– accuracy: 0.9327- val_loss: 0.1466 - val_accuracy: 0.9510

Epoch 5/20

980/980 -1s – loss: 0.1414 – accuracy: 0.9439- val_loss: 0.1399 - val_accuracy: 0.9531

Epoch 6/20

980/980 -1s – loss: 0.1123– accuracy: 0.9561- val_loss: 0.1540- val_accuracy: 0.9378

Epoch 7/20

980/980 -1s – loss: 0.1582 – accuracy: 0.9337 - val_loss: 0.1464 - val_accuracy: 0.9531

Epoch 8/20

980/980 -1s – loss: 0.1509– accuracy: 0.9398- val_loss: 0.1676 - val_accuracy: 0.9367

Epoch 9/20

980/980 -1s – loss: 0.0907 – accuracy: 0.9653- val_loss: 0.1045 - val_accuracy: 0.9582

Epoch 10/20

980/980 -1s – loss: 0.0809 – accuracy: 0.9714- val_loss: 0.1243 - val_accuracy: 0.9561

Epoch 11/20

980/980 -1s – loss: 0.0727 – accuracy: 0.9684- val_loss: 0.1173 - val_accuracy: 0.9592

Epoch 12/20

980/980 -1s – loss: 0.0612 – accuracy: 0.9745- val_loss: 0.1500- val_accuracy: 0.9367

Epoch 13/20

980/980 -1s – loss: 0.0454 – accuracy: 0.9816- val_loss: 0.1683- val_accuracy: 0.9439

Epoch 14/20

980/980 -1s – loss: 0.0633 – accuracy: 0.9786- val_loss: 0.1573 - val_accuracy: 0.9418

Epoch 15/20

980/980 -1s – loss: 0.0809 – accuracy: 0.9714- val_loss: 0.1243 - val_accuracy: 0.9561

Epoch 1/20
Epoch 16/20
980/980 -1s – loss: 0.0727 – accuracy: 0.9684- val_loss: 0.1173 - val_accuracy: 0.9592
Epoch 17/20
980/980 -1s – loss: 0.0672 – accuracy: 0.9745- val_loss: 0.1420 - val_accuracy: 0.9469
Epoch 18/20
980/980 -1s – loss: 0.0662 – accuracy: 0.9816- val_loss: 0.1683 - val_accuracy: 0.9439
Epoch 19/20
980/980 -1s – loss: 0.0391 – accuracy: 0.9878- val_loss: 0.1405 - val_accuracy: 0.9561
Epoch 20/20
980/980 -1s – loss: 0.0173 – accuracy: 0.9949- val_loss: 0.1539 - val_accuracy: 0.9469

Table 4.1: Final accuracy compilation

For step wise accuracy, loss, validation accuracy and validation loss optimization; parameter use Adam, loss function and epochs at most 20. Figure 3.4 shows the function loss and accuracy according to their data process. Five best layer result such as accuracy, loss, and validation accuracy and validation loss shows on Table 4.2.

loop	loss	accuracy	Val_loss	Val_accuracy
15/20	0.0454	0.9816	0.1683	0.9439
16/20	0.0633	0.9786	0.1573	0.9418
17/20	0.0672	0.9745	0.1420	0.9418
18/20	0.0622	0.9724	0.1476	0.9490
19/20	0.0391	0.9878	0.1405	0.9561
20/20	0.0173	0.9949	0.1539	0.9469

Table 4.2: Five best layer result.

The above table shows all the parameter running result of our model. Result of model prediction tested with the 30% data from the storage as the model is used for classifier for five best layers shows the approximately better result along with other layers. Last fifteen to twenty compilation shows the previous figure 3.4 best five result. Plot graph and model analyze optimize result give this accuracy for prediction. Tensorboard use with the keras API with tensorflow using python language. All the plot value are used for the graphical estimation. The more loss minimize the better accuracy obtained.

4.7 Model Test Overview

Another model function are used for preprocess due to validation test. For early testing purpose several data are deploy with the preserve model by implementing. Model feature label are would need to same as previously import method. Such as data directory resize reshape converted one channel etc.

The result predicted with model either 0 or 1 value predict early and late potato blight. An overview of the full testing process in Figure 4.11.

Image Read

Resize

Reshape

.

Load model

Predict with model

| Figure 4.12: Testing Process

4.8 Data Preparation

Data are collected for prepare as the previous approach like image data prepare same size dimension as any class or can be single data retrieval. For this step real collected data are converted as 148X148 dimension with bulkresize Photo's online application same process used here also. For the first image collection from local area data which are separately stored on storage model. Make sure Tested data are not used before.

4.9 Step-by-Step Method

Separately stored data are manually stored before on the cloud storage drive. Image are read here randomly maintain same size 70x70 and reshaping one-dimensional vector. Above all for model finalize which we implemented also deploy in Figure 4.13.

Test Images

148X148

Reshape_{1D}

Resize_{70x70}

Predict model

Figure 4.13: Model Predict process.

The figure 4.12 shows the summary of the predict process. Test images with 148x148 is resize with python opencv library. Reshape use for one vector which applied with our model previously. Parse the test image to test with the predicted model which classify as early blight and late blight.

4.10 Conclusion

This chapter revise practical models of disease detection with classification real image. In addition to the output of every part such as image preprocess resize, reshape training result. Update and modify in accordance loss and accuracy obtained. Optimize with best

layers conv dense layer correspondingly. Full update and final model adjustment with the unseen real data. In the next chapter discuss future work and other modification model with varieties circumstance

CHAPTER 5

CONCLUSION

5.1 Introduction

Conclusion chapter discuss very traditional and manual approach to compare the implemented method and result is better them. There is future scope for this research too. As food one of the basic human need so the deep learning based mobile assisted technology in this era provide a great a role.

5.2 Discussion

Traditional approach for image classification has been based on manual engineered features and use some form of learning algorithm in these feature spaces. These approaches performance of thus depended severely on the underlying predefined features. Feature engineering itself is a complex and tedious process which needs to be revisited all times the problem at hand or the associated dataset changes frequently. This problem occurs in all traditional attempts to detect plant diseases using computer vision as they, image enhancement techniques, and a host of other complex and labor-intensive methodologies. Example of these traditional approaches are SIFT [57] , HoG [58], SURF [59], etc. In addition, traditional approaches to disease classification via machine learning typically focus on a small number of classes usually within a single crop. Examples include a feature extraction and classification pipeline using thermal and stereo images in order to classify tomato powdery mildew against healthy tomato leaves [60] the detection of powdery mildew in uncontrolled environments using RGB images [61]; the use of RGBD images for detection of apple scab [62] the use of fluorescence imaging spectroscopy for detection of citrus huanglongbing [63] the detection of citrus huanglongbing using near infrared spectral patterns [64] and many others. A very recent review on the use of machine learning on plant phenotyping [65] extensively discusses the work in this domain. While nd `eural networks have been used before in plant disease identification [66] (for the classification and detection of Phalaenopsis seedling disease like bacterial soft rot, bacterial brown spot, and Phytophthora black rot), the approach required representing the images using a carefully selected list of texture features before the neural network could classify them. The implementation of the proposed methodology has been carried out stored server or kernel. It consists of around two thousand images belonging to three different classes of potato leaf diseases. Keras, a neural network API written in Python, has been used for the model implementation along tensor flow library. Image classes are mainly two part of namely train dataset and test dataset. In order to increase the dataset, automatic data augmentation techniques has been used by randomly rotating the images by a small amount of 20 degrees, horizontal flipping, vertical and horizontal shifting of images. The optimization was carried out using Adam optimizer with categorical cross entropy as the loss function. Batch size of 20 has been used and the model has been trained for 30 epochs. The initial learning rate has been set to 0.01 and it is reduced by a factor of 0.3 on plateau where the loss stops decreasing. Early stopping has also been used in order to monitor the validation loss and stop the training process once it increases. All the experiments were performed on Intel Core i7-5567 CPU and processor clock speed up to 2.3GHz.

5.3 Future work

The crop disease prevails food insecurity to eat, export import problem etc. But computer vision and penetration of smart phone have paved the way to disease detect. Different learning rates and optimizers could also be used for experimenting with the proposed model as a part of the future work. It could also include experimentation with newer architectures for improving the performance of the model on the train set. Thus, the above mentioned model can be made use of as a decision tool to help and support farmers in detecting the diseases that can be found in the potato leaf. Proposed methodology accuracy 94% optimization can make an accurate detection of the leaf diseases abridges diagnosis effort. By the effective of this methodology mobile based further detection bring to image based disease detect handier to use.

REFERENCES

- [1] **Molla M(2019) Potato glut badly hurts growers, thedailystar.net, pp. 10-15, [https://www.thedailystar.net/backpage/potato-production-in-bangladesh-glut-25 May 2019](https://www.thedailystar.net/backpage/potato-production-in-bangladesh-glut-25-May-2019).**
- [2] Jahangir Z(2014), Export product of mega group: fresh potato, <http://www.megagroup.com.bd> 2014.
- [3] Pam Mercure P(2012), Early Blight and Late Blight of Potato. Dissertation University of Guelph.
- [4] M. Hossain, T.K. Dey, M. Iqbal Hossain, S.N. Begum, M.S. Kadian(2008), Research management in Bangladesh, Acta Hort. 834, 175-186
DOI: 10.17660/ActaHortic.2009.834.19.
- [5] Geert kessel, mushfiqul islam, mohammad maroof, maniruzzaman mukul, aby syehengsdijk(2019) Geodata for the control of potato late blight in Bangladesh.<https://agro.au.dk/fileadmin/euroblight/Workshops/York/Presentations/p57-62.pdf>. Accessed MAY 2019.
- [6] MSU, 2013.
- [7] Mitchell J. Bauske, Andrew P. Robinson, Neil C. Gudmestad(2018), Early Blight in (NDSU) Extensions, pp1892. <https://www.ag.ndsu.edu/publications/crops/early-bl>
- [8]

**Gerrit Polder, Pieter M. Blok, Hendrik Jan M. van der Wolf, and Jan Kamp, I
Detection in Seed Potatoes Using Deep**

-
- [1] Molla M(2019) Potato glut badly hurts growers, [thedailystar.net](https://www.thedailystar.net/backpage/potato-production-in-bangladesh-glut), pp. 10-15, <https://www.thedailystar.net/backpage/potato-production-in-bangladesh-glut>, 25 May 2019.
-

spectral Images. Front Plant Sci. 2019; 10.3389/fpls.2019.00209.

- [9] EHLER, Lester E(2006), Integrated pest management (IPM): Definition, historical and other IPM. Vol 62, No 9, pp 787-789. Wiley, Chichester United Kingdom.
- [10] ITU International Telecommunication Union (2015), ICT Facts and Figures the W Geneva Switzerland 2015. <https://www.itu.int/en/ITU-D/Statistics/Documents/publications/ITU-D-Statistics-2015>
- [11] Tai, A.P.K., M. Val Martin and C.L. Heald (2014), Threat to future global food security: pollution, 2014. doi: 10.1038/nclimate2317. <https://globalchange.mit.edu/publications>
- [12] J. Msuya et al (2019), Report of the Plenary of the Intergovernmental Science-Policy Platform on Biodiversity and Ecosystem Services, 19 April 2019. <https://ipbes.net/event/ipbes-7-plenary>.
- [13] Identification of rice diseases using deep convolutional neural networks
- [14] Y. Lu, Shujuan Yi, Nianyin Zeng, Y. Liu, Y. Zhang (2017) Identification of rice diseases using deep convolutional neural networks, ELSEVIER, pp. 20-40, 2017. DOI:10.1016/j.neucom.2017.06.023.
- [15] Jihen Amara, Bassem Bouaziz (2017), A Deep Learning-based Approach for Banana Disease Detection, pp. 79–89. <https://pdfs.semanticscholar.org/adae/9446cb66eaa6645dca78fd81b21d43e>
- [16] David P. Hughes & Marcel Salathe (2015) An open access repository of images, 2015. <https://arxiv.org/ftp/arxiv/papers/1511/1511.08060.pdf>
- [17] Luc Van Gool, Christopher K. I. Williams · John Winn, Andrew Zisserman John Winn (2015) A large-scale recognition of visual object classes (voc) challenge. Int. J. Comput. Vis. 88, 303–338.
- [18] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean M. Berneid, Michael Bernstein, Alexander C. Berg, Li Fei-Fei (2015), ImageNet large scale visual recognition challenge. Comput. Vis. 115, 2015. <https://arxiv.org/abs/1409.0575>

-
- [1] **Molla M(2019) Potato glut badly hurts growers, thedailystar.net, pp. 10-15, <https://www.thedailystar.net/backpage/potato-production-in-bangladesh-glut> 25 May 2019.**
-
- [19] U. Mokhtar, N. El-Bendary, Aboul Ella Hassenian, E. Emary, M. Mahmoud, Hesh Detection of Rice Leaves Diseases, Springer, 2015. DOI:10.1007/978-3-319-1131
- [20] Sachin D. Khirade, A. B. Patil(2015), Plant Disease Detection,In: 2015 Internatio
- [21] N. S. Sabah Bashir(2012), Remote Area Plant Disease Detection Using Image Pro Communication Engineering,vol. 2, no. DOI:10.1109/ICCUBEA.2015.153
- [22] Xi-Jin Zhang, Yi-Fan Lu, Song-Hai Zhang(2016), Multi-Task Learning for Food I Deep,journal of computer science and technology 31(3): 489–500, May 2016.
- [23] Mobility, Ericsson: Interim Update: Ericsson Mobility Report, 2015.
- [24] E.-C. Oerke, Crop losses to pests, The Journal of Agricultural Science, vol. 144(01 <https://doi.org/10.1017/S0021859605005708>
- [25] Li Deng (2014) A tutorial survey of architectures, algorithms, and applications for Signal and Information Processing. DOI: <https://doi.org/10.1017/atsip.2013.9>
- [26] I. Arel, Derek C. Rose, T. Karnowski(2016), Deep Machine Learning - A New IEE DOI:10.1109/MCI.2010.938364.
- [27] B. B. a. A. A. Jihen Amara et al, "A Deep Learning-based Approach for Banana L BTW 2017 – Workshopband, 2017.
- [28] LeCun et al(1989), Backpropagation applied to handwritten zip code recognition,`
- [29] *Yann LeCun, Leon Bottou, Yosuha Bengio and Patrick Haffner (1990), a neural ne machine-printed character recognition called LeNet-5.*
- [30] M. Rizwan(2020), LeNet-5 – A Classic CNN Architecture available at <https://engr architecture/>," 3 March 2020.

-
- [1] **Molla M(2019) Potato glut badly hurts growers, thedailystar.net, pp. 10-15, <https://www.thedailystar.net/backpage/potato-production-in-bangladesh-glut> 25 May 2019.**
-

- [31] T. M. Prajwala, Alla Pranathi, Kandiraju Sai Ashritha, Nagaratna B. Chittaragi, Sh Disease Detection using Convolutional," Proceedings of 2018 Eleventh International Computing (IC3), 2-4 August 2018.
- [32] Yann LeCun et al, Leon Bottou, Yoshua Bengio, and Patrick Haffner (1998), "Gradient Descent for Linear Support Vector Machines," Proceedings of the IEEE Conference on Neural Networks for Handwriting Recognition and Recognition, ROC. OF THE IEEE, NOVEMBER 1998.
- [33] H. S. Seung and H. Sompolinsky(1992), Statistical mechanics of learning from examples, 1992.
- [34] **Vladimir Vapnik et al (1994), Measuring the VC-dimension of a Function Class. Machine. Doi:10.1.1.32.8584.**
- [35] Corinna Cortes, L. D. Jackel, Sara A. Solla, Vladimir Vapnik, John S. Denker (1993) Rate of Convergence, NIPS, 1993.
- [36] Vapnik, Vladimir(1998), "The Nature of Statistical Learning Theory. Springer , 1998.
- [37] William H. Press, Saul A. Teukolsky, William T. Vetterling, Brian P. (1986) Numerical Recipes in C: The Art of Scientific Computing Cambridge University Press, Cambridge, 1986.
- [38] Mohanty et al (2016), Using Deep Learning for Image-Based Plant Disease Detection, September 2016. <https://doi.org/10.3389/fpls.2016.01419>.
- [39] Jae. S. Lim (1990), Two-dimensional signal and image processing, Englewood Cliffs, NJ: Prentice-Hall.
- [40] Harrison(2018), Deep Learning basics with Python, TensorFlow and Keras, access <https://pythonprogramming.net/loading-custom-data-deep-learning-python-tensorflow/>
- [41] G. Cheng, P. Zhou and J. Han(2016), Learning Rotation-Invariant Convolutional Neural Networks for VHR Optical Remote Sensing Images, vol. 54, no. 12, pp. 7405 - 7415, 2016.

-
- [1] **Molla M(2019) Potato glut badly hurts growers, thedailystar.net, pp. 10-15, <https://www.thedailystar.net/backpage/potato-production-in-bangladesh-glut> 25 May 2019.**
-
- [42] G. Cheng, J. Han, P. Zhou and D. Xu(2019), Learning Rotation-Invariant and Fish Networks for Object Detection, vol. 28, no. 1, January 2019.
- [43] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Vanhoucke, Andrew Rabinovich(2015) Going Deeper with Convolutions, <https://arxiv.org/abs/1504.00562>
- [44] Jiuxiang Gu, Zhenhua Wang, Jason Kuen, Lianyang Ma, Amir Shahroudy, Bing Si Gang Wang, Jianfei Cai, Tsuhan Chen(2017), Recent advances in convolutional neural networks, Elsevier 2017.
- [45] Yinhui Deng, Yuanyuan Wang, Senior Member, IEEE, and Ping Chen (2008)Auto Syndrome from ultrasound images,Conf. Proc., vol. 2008, p. pp. 4772–4775, 2008
- [46] Palak Mehrotra ; Jyotirmoy Chatterjee ; Chandan Chakraborty ; Biswanath Ghosh Automated Screening of PCOS using Machine learning technique palak., IEEE International Conference on Bioinformatics and Biomedical Engineering (ICBBE) 2017
- [47] E. Allibhai(2018), Keras tutorial – build a convolutional neural network.,Accessed 12 April 2020. <https://www.kdnuggets.com/2018/04/building-convolutional-neural-network-keras.html>
- [48] Ji Young Lee, Franck Dernoncourt(2016), Sequential Short-Text Classification with Recurrent Neural Networks, in conference paper at NAACL, 2016. 515–520 <https://www.aclweb.org/anthology/N16-1062> DOI:10.18653/v1/N16-1062
- [49] Getting started with the Keras Sequential model. Accessed 12 april 2020. <https://keras.io/getting-started/sequential-model-primer/>
- [50] Alex Krizhevsky,Ilya Sutskever,Geoffrey E. Hinton(2012), Imagenet classification with deep convolutional neural networks,In: Advances in neural information processing systems,pp. 1097–1105.
- [51] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Vanhoucke, Andrew Rabinovich(2015) Going Deeper with Convolutions, <https://arxiv.org/abs/1504.00562>
- [52] Yann Lecun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, L. Jackel handwritten.,IEEE, p. 541–551, 1989.

-
- [1] **Molla M(2019) Potato glut badly hurts growers, thedailystar.net, pp. 10-15, <https://www.thedailystar.net/backpage/potato-production-in-bangladesh-glut> 25 May 2019.**
-

- [53] T. M. Prajwala, Alla Pranathi, Kandiraju Sai Ashritha, Nagaratna B. Chittaragi, Sh Disease Detection using Convolutional, Proceedings of 2018 Eleventh Internation (IC3) in IEEE, 2018. DOI:10.1109/IC3.2018.8530532.
- [54] T. M. Prajwala, Alla Pranathi, Kandiraju Sai Ashritha, Nagaratna B. Chittaragi, Sh Disease Detection using Convolutional Neural Networks.,"
- [55] TensorFlow(2019), Using TensorBoard in Notebooks.,available at https://www.tensorflow.org/tensorboard/tensorboard_in_notebooks. Accessed May
- [56] Harrison(2018), Deep Learning basics with Python, TensorFlow and Keras, Analy 2018.
- [57] David G. LOWE(2004), Distinctive Image Features from Scale-Invariant K Vision), 91–110, vol. 60, no. 2.
- [58] N. Dalal and B. Triggs(2005), Histograms of Oriented Gradients for Human Detec
- [59] Herbert Bay , Andreas Ess , Tinne Tuytelaars , Luc Van Gool (2008), Speeded-Up 346–359.
- [60] Shan-e-Ahmed Raza; Prince, Gillian; P. Clarkson, John; M. Rajpoot, Nasir (2015) Plants Using Thermal and Stereo Visible Light Images, 2015. <https://doi.org/10.13>
- [61] Deny Lizbeth Hernández-Rabadán,Fernando Ramos-Quintana and Julian Guerrero Bayesian Classifier for Segmenting," Hindawi Publishing Corporation, Oct 6, 201
- [62] Chéné Y, Rousseau D, Lucidarme P, Bertheloot J, Caffier V, Morel P, Belin É, Cha depth camera for 3d phenotyping of entire," Computers and Electronics in agricult 10.1016/j.compag.2011.12.007.
- [63] Caio B. Wetterich, Ratnesh Kumar, Sindhuja Sankaran, José Belasque Junior,Reza comparative study on application of computer vision and fluorescence imaging sp citrus disease in the USA and Brazil,," Journal of Spectroscopy, 2013. Volume 201

-
- [1] **Molla M(2019) Potato glut badly hurts growers, thedailystar.net, pp. 10-15, <https://www.thedailystar.net/backpage/potato-production-in-bangladesh-glut> 25 May 2019.**
-

- [64] Francisco Garcia-Ruiz , Sindhuja Sankaran, Joe Mari Maja, Won Suk Lee c, Jespe "Comparison of two aerial imaging platforms for identification of," Computer and 2013. <http://dx.doi.org/10.1016/j.compag.2012.12.002>

- [65] Arti Singh,Baskar Ganapathysubramanian,,Asheesh Kumar Singh,Soumik Sarkar Throughput Stress Phenotyping in Plants," 2015. VOLUME 21, ISSUE 2, P110-1: DOI:<https://doi.org/10.1016/j.tplants.2015.10.015>

- [66] Kuo-YiHuang(2007) "Application of artificial neural network for detecting Phalae texture features ELSEVIER - Computer and Electronics in Agriculture, Volume 57 <https://doi.org/10.1016/j.compag.2007.01.015>

- [67] Fariahrasha(2019), Potatoes nine severe disease and prevention., Accessed 2 July :

- [68] Uday Pratap Singh ; Siddharth Singh Chouhan ; Sukirty Jain ; Sanjeev Jain (2019) for the Classification of Mango Leaves Infected by Anthracnose Disease, IEEE Ac 2019 ,10.1109/ACCESS.2019.2907383.

- [69] M. Sornam, Kavitha Muthusubash, V. Vanitha(2017), A Survey on Image Classific Convolutional Neural Network Architecture, Computer Science 2017 Ninth Intern Computing (ICoAC),DOI:10.1109/ICOAC.2017.8441512.

- [70] T. K. Dey, M. Hossain, M. S. Kadian, S. Hossain, M. Bonierbale, A. A. Mahmud(management of potato late blight in Bangladesh.," Potato Journal , vol. 37 No.3/4,

- [71] Willie Kirk (2014), Fungicides for late blight control in potatoes, Michigan State U Soil and Microbial Sciences, 16 July 2014. <https://www.canr.msu.edu/news/>

- [72] Z.m. zahangir (2014), Export product of mega group: fresh potato. <http://www.me> February 15, 2014

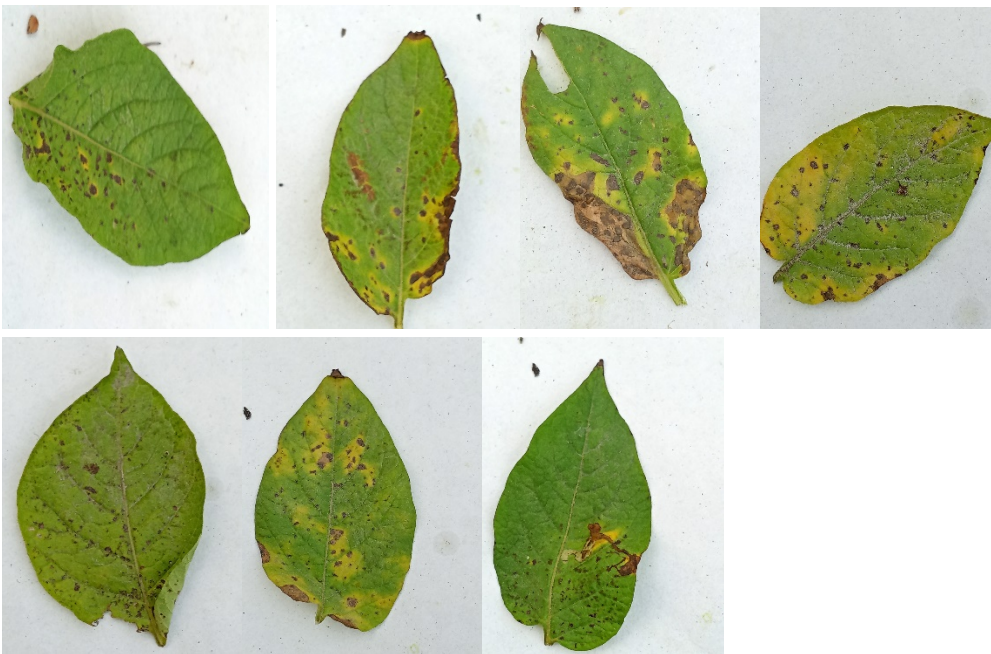
- [73] Mehi Lal, Sanjeev Sharma, Saurabh Yadav and Santosh Kumar(2018),Managemen reviewed chapterDOI: 10.5772/intechopen.72472,Accessed June 6th 2018.

-
- [1] Molla M(2019) Potato glut badly hurts growers, thedailystar.net, pp. 10-15, <https://www.thedailystar.net/backpage/potato-production-in-bangladesh-glut-25-May-2019>.
-

- [74] Mahadhan SMARTEK(2017) Potato: Diseases and Symptoms, <https://mahadhan.c> Oct 9,2017.

- [75] E. Allibhai(2018), Keras tutorial build a convolutional neural network, Available: [convolutional-neural-network-cnn-in-keras-329fbbadc5f5](https://www.kaggle.com/allibhai/convolutional-neural-network-cnn-in-keras-329fbbadc5f5). Accessed 16 October 2018.
-

A. Collected data correspondingly early blight data:



And late blight data:



Appendix B:

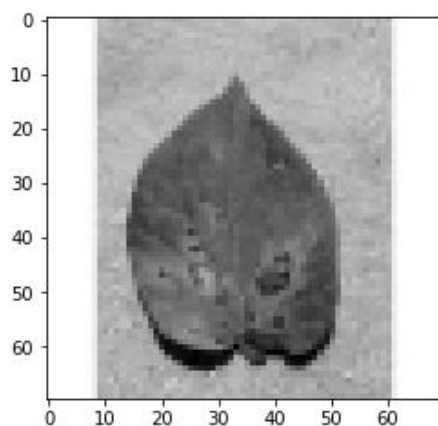
Code snippet: Path for Class merge and run as single directory.

```
DATADIR = "/content/drive/My Drive/Colab Notebooks/Thesis"
CATEGORIES = ["Potato_Early_blight", "Potato_Late_blight"]
i = 0
for category in CATEGORIES:
    path = os.path.join(DATADIR,category)
    for img in os.listdir(path):
        print(img)
        i=i+1
        if i==5:
            break
    img_array = cv2.imread( os.path.join(path,img),cv2.IMREAD_GRAYSCALE)
    plt.imshow(img_array,cmap="gray")
    plt.show()
    #break
break
```

Reshape Data Snippet:

```
IMG_SIZE = 70
```

```
new_array = cv2.resize(img_array, (IMG_SIZE, IMG_SIZE))  
plt.imshow(new_array, cmap='gray')  
plt.show()
```



Two Model summery snippet prior best Model: 1) Small model with 64 conv layer

```
1-conv-64-nodes-1-dense-1583608673  
Model: "sequential_121"
```

Layer (type)	Output Shape	Param #
conv2d_233 (Conv2D)	(None, 48, 48, 64)	640
activation_424 (Activation)	(None, 48, 48, 64)	0
max_pooling2d_233 (MaxPoolin	(None, 9, 9, 64)	0
flatten_119 (Flatten)	(None, 5184)	0
dense_229 (Dense)	(None, 64)	331840
activation_425 (Activation)	(None, 64)	0
dense_230 (Dense)	(None, 1)	65
activation_426 (Activation)	(None, 1)	0
Total params: 332,545		
Trainable params: 332,545		
Non-trainable params: 0		
Train on 1372 samples, validate on 588 samples		

2) Best model with 256 Conv l ayer

Layer (type)	Output Shape	Param #
conv2d_14 (Conv2D)	(None, 68, 68, 256)	2560
activation_23 (Activation)	(None, 68, 68, 256)	0
max_pooling2d_14 (MaxPooling)	(None, 34, 34, 256)	0
conv2d_15 (Conv2D)	(None, 32, 32, 256)	590080
activation_24 (Activation)	(None, 32, 32, 256)	0
max_pooling2d_15 (MaxPooling)	(None, 16, 16, 256)	0
flatten_7 (Flatten)	(None, 65536)	0
dense_14 (Dense)	(None, 64)	4194368
activation_25 (Activation)	(None, 64)	0
dense_15 (Dense)	(None, 1)	65
activation_26 (Activation)	(None, 1)	0